Grado en Ingeniería Informática

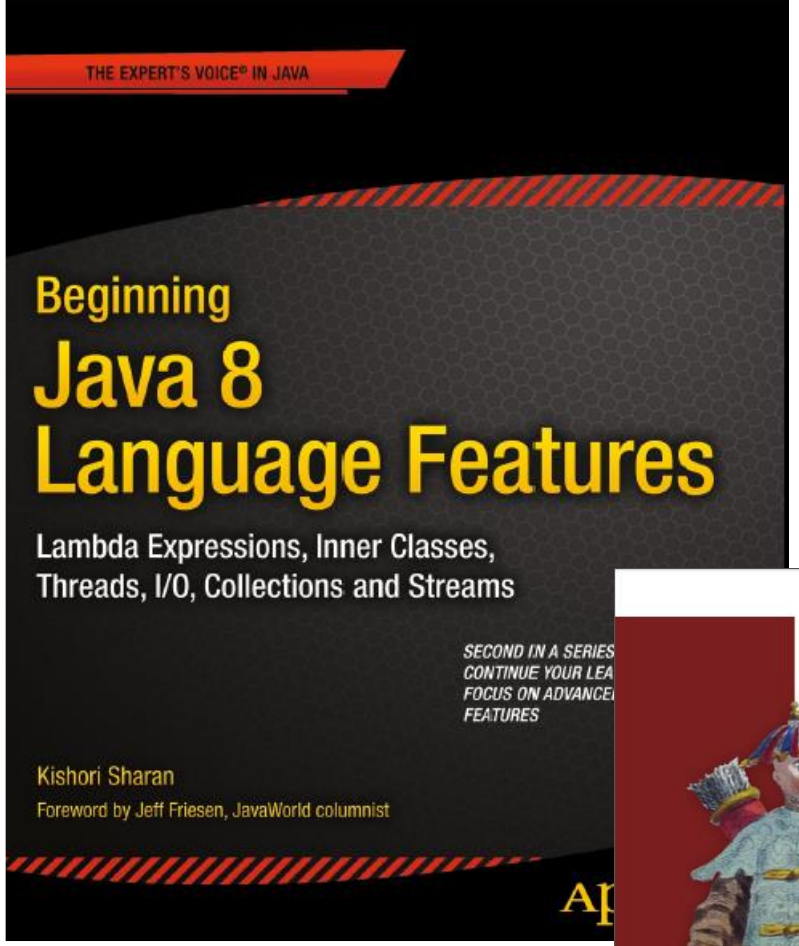
Tecnologías de Desarrollo de Software

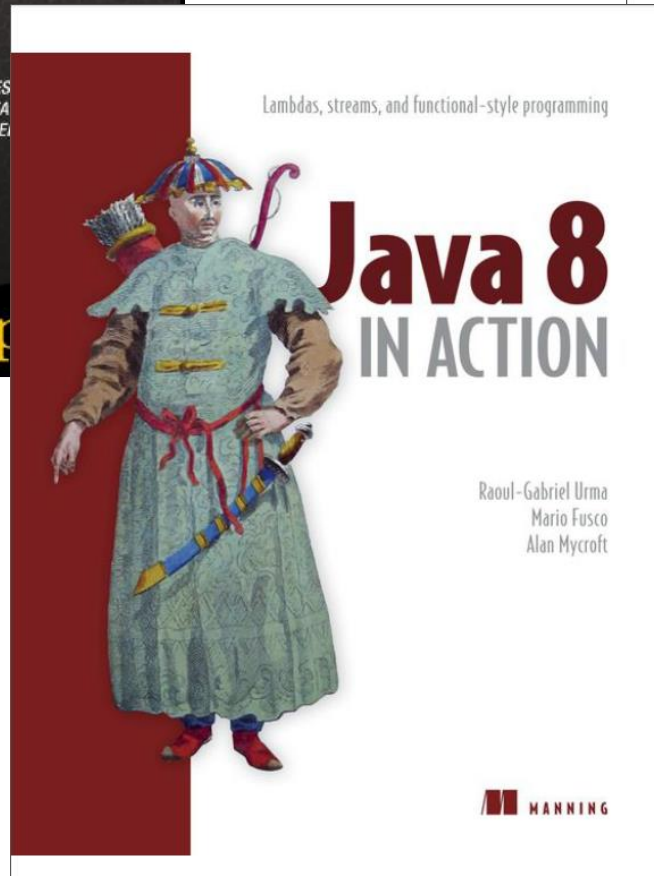
Tema 3

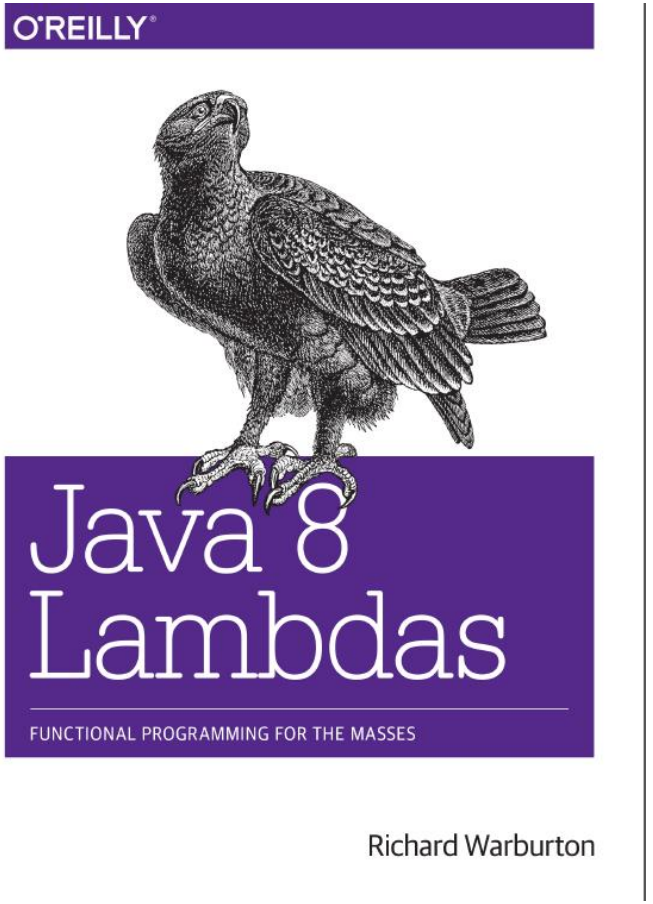
Algunos conceptos avanzados de programación orientada a objetos

Contenidos

- **Clases anidadas**
  - **Anidadas estáticas e Internas**
- Programación Funcional:
  - Closures (function object)
  - Expresiones Lambda en Java 8
  - Iteradores en Java 8 : Streams
    - Patrón Iterador: Iteradores externos e internos
  - Tipo Optional en Java 8
- Metaprogramación e Introspección: Reflexión en Java


THE EXPERT'S VOICE® IN JAVA
Beginning
Java 8
Language Features
Lambda Expressions, Inner Classes,
Threads, I/O, Collections and Streams
SECOND IN A SERIES
CONTINUE YOUR LEA
FOCUS ON ADVANCE
FEATURES
Kishori Sharan
Foreword by Jeff Friesen, JavaWorld columnist


Lambdas, streams, and functional-style programming
Java 8
IN ACTION
Raoul-Gabriel Urma
Mario Fusco
Alan Mycroft
MANNING


O'REILLY
Java 8
Lambdas
FUNCTIONAL PROGRAMMING FOR THE MASSES
Richard Warburton

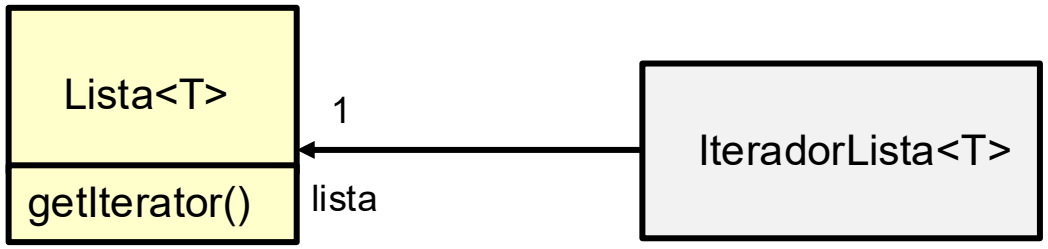
Clases anidadas

- Algunos lenguajes OO, como C++, C# y Java, permiten anidar la definición de una clase dentro de otra clase cuando sólo es necesaria en ese ámbito.
- En Java se incluyó, además, el mecanismo de clases internas para parametrizar clases con funciones.
- En el caso de Java hay 4 mecanismos de anidamiento de clases:
  - Clases **anidadas estáticas**
  - Clases **internas**
    - Clases miembro, clases anónimas, y clases locales.



- Nodos no necesitan acceder a la lista de la que forman parte.
- Nodos se definen para un tipo de lista.

Clase **anidada estática**



- Un iterador necesita acceder a la lista que recorre.
- Iterador se define para un tipo de lista.

Clase **interna**

Clases anidadas en Java

- Clases anidadas **estáticas**
  - Una clase se declara en el **ámbito de otra clase lógicamente relacionada.**

```
public class Externa {..
   static class AnidadaEstatica {..}
}
```

- Un clase anidada estática tiene visibilidad sobre los miembros de la clase que la engloba (incluso los privados), y al revés.

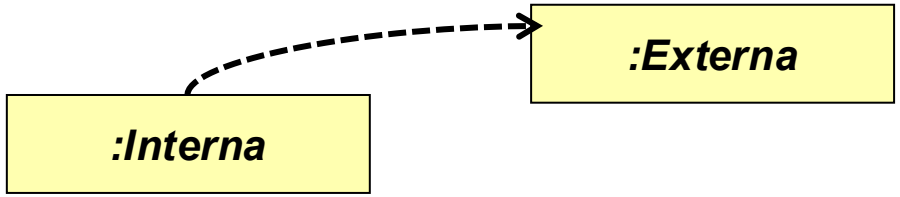
```
public class Externa {..
        private int atrib;
        public static class Anidada {
                void m1(Externa pExt) {
                                pExt.atrib = 10;
                }
        }
}
```

Clases anidadas en Java: Ejemplo

```
// implementación de pilas antes de incluir clases
   genéricas
public class Stack {
   private Node head;
   private static class Node {
        private Object value;
        private Node next;
        public Node(Object v, Node s){
            value = v;  next = s;
        }
   }
   public void push (Object obj){
        head = new Node (obj,head);
   }
   public Object pop (){
        Object first = head.value;
        head = head.next;
        return first;
   }
}
```

Clases internas en Java

- También relación lógica entre la clase externa e interna, pero ahora existe una **relación entre instancias de dos clases**.



- Una **instancia de una clase interna:**
  - Debe existir en el contexto de una instancia de su clase externa y tiene acceso directo a sus campos y métodos.
  - Creada en algún método de instancia de la clase externa.
- **Útiles** para **parametrizar una clase por una acción a invocar**: (callback o patrón *Command*)

A menudo, **implementan una interfaz**

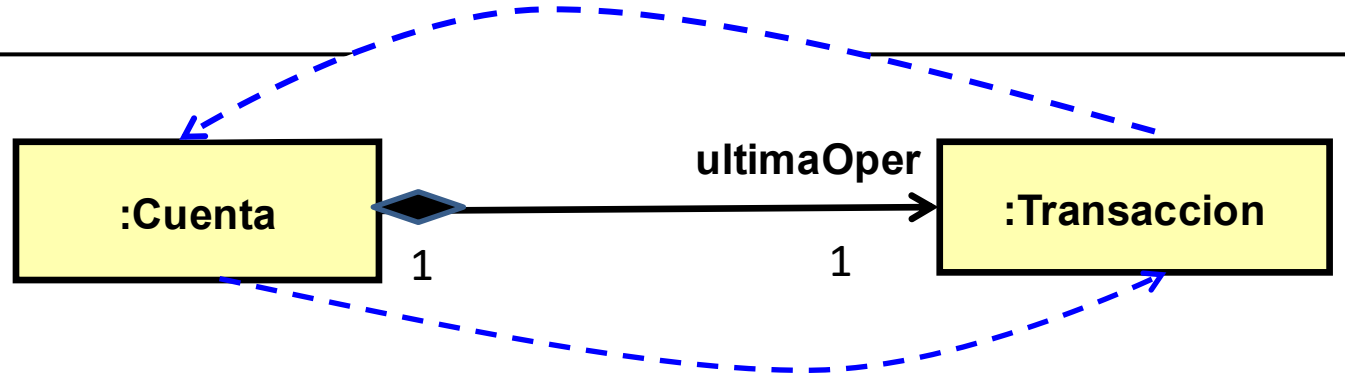
```
class Externa {..
   class Interna implements Interfaz {..}
}
```

Clases interna: Ejemplo “Cuenta bancaria”

(Tomado de “**The Java Programming Language”,** K. Arnold et al. , 3ª Edición)

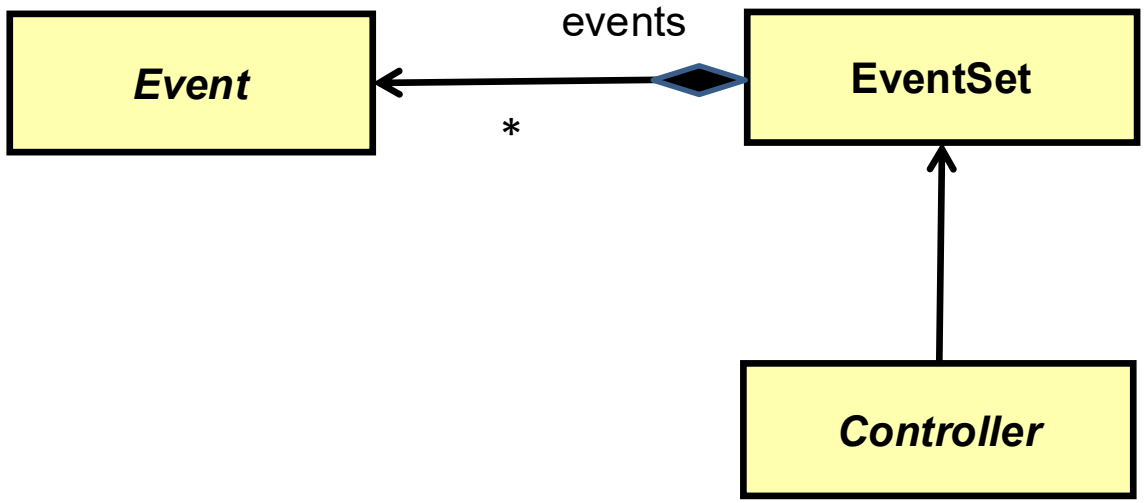
```
class Cuenta{..
   private long codigo;
   private long saldo;
   private Transaccion ultimaOper;

    public class Transaccion {
        private String descripcion;
        private long cantidad;
        ...
        public String toString(){
          return codigo + ": "+ descripcion+"  "+cantidad;
        }
    }
   public void reintegro(long cantidad){
       saldo = saldo - cantidad;
       ultimaOper= this.new Transaccion("reintegro",cantidad);
   }
}
```



Clases internas: Ejemplo *framework "manejo de eventos de dispositivos"*

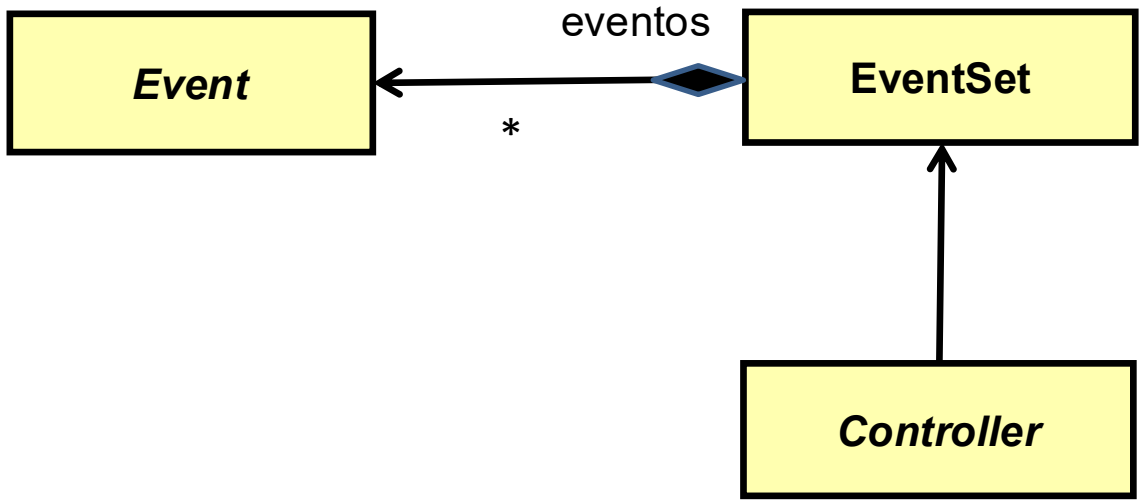
(Tomado de "Thinking in Java", Bruce Eckel, 1ª Edición)



- Para crear una aplicación concreta es necesario crear:
  - una **subclase** de `Controller` y
  - una **subclase** de `Event` para cada tipo de evento.
- Estas subclases **necesitan actualizar estado del controlador creado**.

```
abstract public class Event {
   private long evtTime;
   public Event (long eventTime) {
                        this.evtTime = eventTime;
   }
   public boolean ready() {
        return System.currentTimeMillis() >= evtTime;
   }
   abstract public void action();
   abstract public String description();
}
class EventSet {
   private Event[] events = new Event[100];
   private int index = 0;
   private int next = 0;
   public void add() {...}
   public Event getNext () {...}
   public void removeCurrent() {...}
}
```

Clases internas: Ejemplo *framework "manejo de eventos de dispositivos"*




```
abstract public class Controller {
   private EventSet es = new EventSet();
   public void addEvent(Event e) {
                es.add(e);
   }
   public void run() {
      Event ev;
      while (ev = es.getNext()) != null )
         if (ev.ready()) {
           ev.action();
           System.out.println(ev.description());
           es.removeCurrent();
         }
   }
}
```

- Para crear una aplicación concreta es necesario crear:
  - una **subclase** de `Controller` y
  - una **subclase** de `Event` para cada tipo de evento.
- Estas subclases **necesitan actualizar estado del controlador creado**.

Clases internas: Ejemplo *framework "manejo de eventos de dispositivos"*

```
public class ControladorInvernadero
                              extends Controller {
   private boolean luz = false;
   private boolean agua = false;

   private class LuzOn extends Event {
        public LuzOn(long eventTime) {super(eventTime);}
        public void action() { luz = true; }
        public void description() {
                        return "luz encendida"; }
   }

    private class LuzOff extends Event {...}
   private class AguaOn extends Event {...}
   private class AguaOff extends Event {...}
   ...
}
```

Las **subclases** de `Event` se crean como **clases internas** de la clase que hereda de `Controller`.

Clases Internas : Ejemplo LinkedList<E>

```
public class LinkedList<E>
    extends AbstractSequentialList<E>
    implements List<E>, Deque<E>, Cloneable, java.io.Serializable {..

    transient int size = 0; transient Node<E> first; transient Node<E> last;
```

```
private static class Node<E> {
        E item;
        Node<E> next;
        Node<E> prev;
        Node(Node<E> prev, E element, Node<E> next) {
            this.item = element; this.next = next; this.prev = prev;
        }
}
```

```
private class ListItr implements ListIterator<E> {
    private Node<E> lastReturned;
    private Node<E> next;
    private int nextIndex;
    ListItr(int index) {
        next = (index == size) ? null : node(index);
        nextIndex = index;
    }
    public boolean hasNext() {...
        return nextIndex < size;
    }
```

Clases Internas: Ejemplo de manejo de eventos de JavaFX

```java
public class BotonClaseInterna extends Application {

    @Override
    public void start(Stage primaryStage) {
        Button boton = new Button("Púlsame");

        // Asignamos el manejador usando una clase interna
        boton.setOnAction(new ManejadorBoton());

        StackPane root = new StackPane();
        root.getChildren().add(boton);

        Scene scene = new Scene(root, 300, 200);
        primaryStage.setTitle("Ejemplo Clase Interna");
        primaryStage.setScene(scene);
        primaryStage.show();
    }

    // Clase interna nombrada que implementa EventHandler
    private class ManejadorBoton implements EventHandler<ActionEvent> {
        @Override
        public void handle(ActionEvent event) {
            System.out.println("Botón pulsado (clase interna).");
        }
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```

Clases Internas Anónimas: Ejemplo de manejo de eventos de JavaFX

- Una clase interna anónima se crea **“al vuelo”** en el punto en que se necesita un objeto de una clase que implemente una interfaz o clase abstracta.
  - Se usa normalmente como un ***object function***
  - Se crea si sólo se necesita en un punto del programa y debería ser pequeña

```java
public class BotonClaseAnonima extends Application {

    @Override
    public void start(Stage primaryStage) {
        Button boton = new Button("Púlsame");

        // Asignamos el manejador con una clase interna anónima
        boton.setOnAction(new EventHandler<ActionEvent>() {
            @Override
            public void handle(ActionEvent event) {
                System.out.println("Botón pulsado (clase interna anónima).");
            }
        });

        StackPane root = new StackPane();
        root.getChildren().add(boton);

        Scene scene = new Scene(root, 300, 200);
        primaryStage.setTitle("Ejemplo Clase Interna Anónima");
        primaryStage.setScene(scene);
        primaryStage.show();
    }
```

Clases anónimas. Restricción acceso variables

- Sólo pueden usar variables del método en que se ha definido si las variables han sido declaradas como **finales.**

```
final String nombre = getNombre();

boton.setOnAction(new EventHandler<ActionEvent>() {
        public void handle (ActionEvent event) {
                System.out.println("hola " + nombre);
        }
});
```

En Java, una clase anidada es una clase definida dentro de otra clase. Las clases anidadas se dividen en dos categorías: clases anidadas estáticas y clases internas (o no estáticas). Aquí están las diferencias clave entre estas dos:

1. **Acceso al Contexto Externo**:
   - **Clase Interna**: Una clase interna no estática tiene acceso a todos los miembros (incluidos los privados) de la clase exterior en la que está definida. Esto se debe a que cada instancia de una clase interna está asociada con una instancia de la clase exterior.
   - **Clase Anidada Estática**: Una clase anidada estática no tiene acceso a los miembros de la clase exterior a menos que se le pase una referencia explícita a una instancia de la clase exterior.
2. **Modificador Static**:
   - **Clase Interna**: No lleva el modificador `` `static` ``.
   - **Clase Anidada Estática**: Lleva el modificador `` `static` ``.
3. **Instanciación**:
   - **Clase Interna**: Para instanciar una clase interna, primero debe existir una instancia de la clase exterior.
   - **Clase Anidada Estática**: Puede ser instanciada sin una instancia de la clase exterior.
4. **Referencia al Objeto Exterior**:
   - **Clase Interna**: Mantiene una referencia al objeto de la clase exterior, lo que puede ser una consideración importante en términos de gestión de memoria.
   - **Clase Anidada Estática**: No mantiene ninguna referencia al objeto de la clase exterior, lo que puede ser más eficiente en términos de memoria.
5. **Uso**:
   - **Clase Interna**: A menudo se utiliza cuando hay una relación fuerte y lógica entre la clase interna y la clase exterior, y se necesita acceso a los miembros de la clase exterior.
   - **Clase Anidada Estática**: Se utiliza cuando la clase anidada no necesita acceder a los miembros de la clase exterior, o cuando desea separar la clase anidada de la instancia de la clase exterior.

Contenidos

- Clases anidadas
  - Anidadas estáticas e Internas
- **Programación Funcional:**
  - **Closures (function object)**
  - Expresiones Lambda en Java 8
  - Iteradores en Java 8 : Streams
    - Patrón Iterador: Iteradores externos e internos
  - Tipo Optional en Java 8
- Metaprogramación e Introspección: Reflexión en Java

Programación Funcional: Closure (Function object)

- En programación funcional, función anónima que puede ser tratada como un objeto (registrado en una variable y pasado de un contexto a otro).
- Puede ser invocada en un contexto diferente al que se define sin conocer su nombre.
- Lleva ligada el entorno (variables) en el que se define y puede acceder a ellas desde otro contexto.
- Lenguajes OO tipados dinámicamente como Smalltalk, Python y Ruby proporcionan closures **(bloques de código),** también disponibles en JavaScript y otros lenguajes OO.
- Disponibles en Java a partir de Java 8 (lanzado en 2014, 20 años después de aparecer Java)

Closure en Java

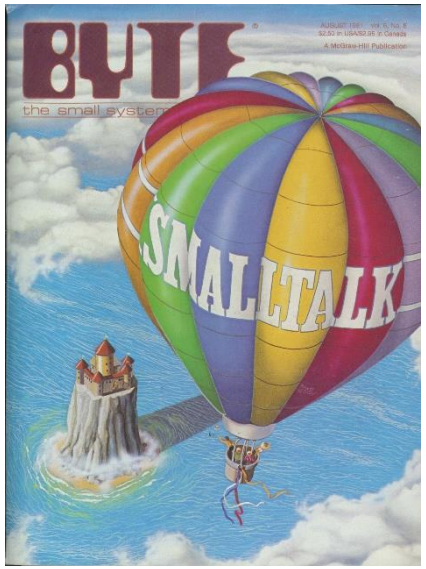
Clases anónimas de Java con un método proporcionan una forma **limitada** de closure.

- La función **es un método con nombre no anónima** y puede usar variables del método en que se definen si son declaradas **`final`**, y **campos de la clase que la engloba.**

```
final String nombre = getNombre();
boton1.addActionListener(new ActionListener() {
        public void actionPerformed(ActionEvent event) {
                System.out.println("hola " + nombre);
        }
});
```

Expresiones lambda de Java 8 **sí** son closures con la restricción de que a las variables del método en que se definen sólo se les puede asignar valor una vez (*effectively final*) para ser usadas en la expresión lambda.

```
String nombre = getNombre();
boton1.addActionListener(event ->
                        System.out.println("hola " + nombre));
```

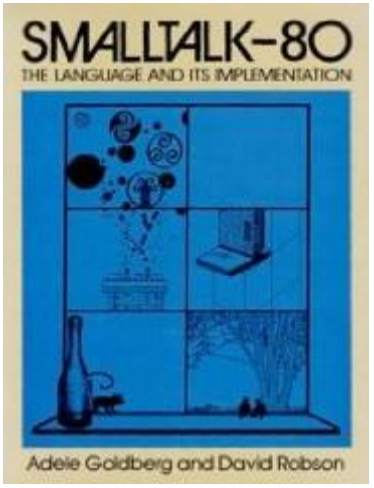


¡¡ Hace más de 40 años se podía escribir este código con Smalltalk … mucho antes de aparecer Java !!

“`colCuentas` es una colección de objetos `Cuenta`”

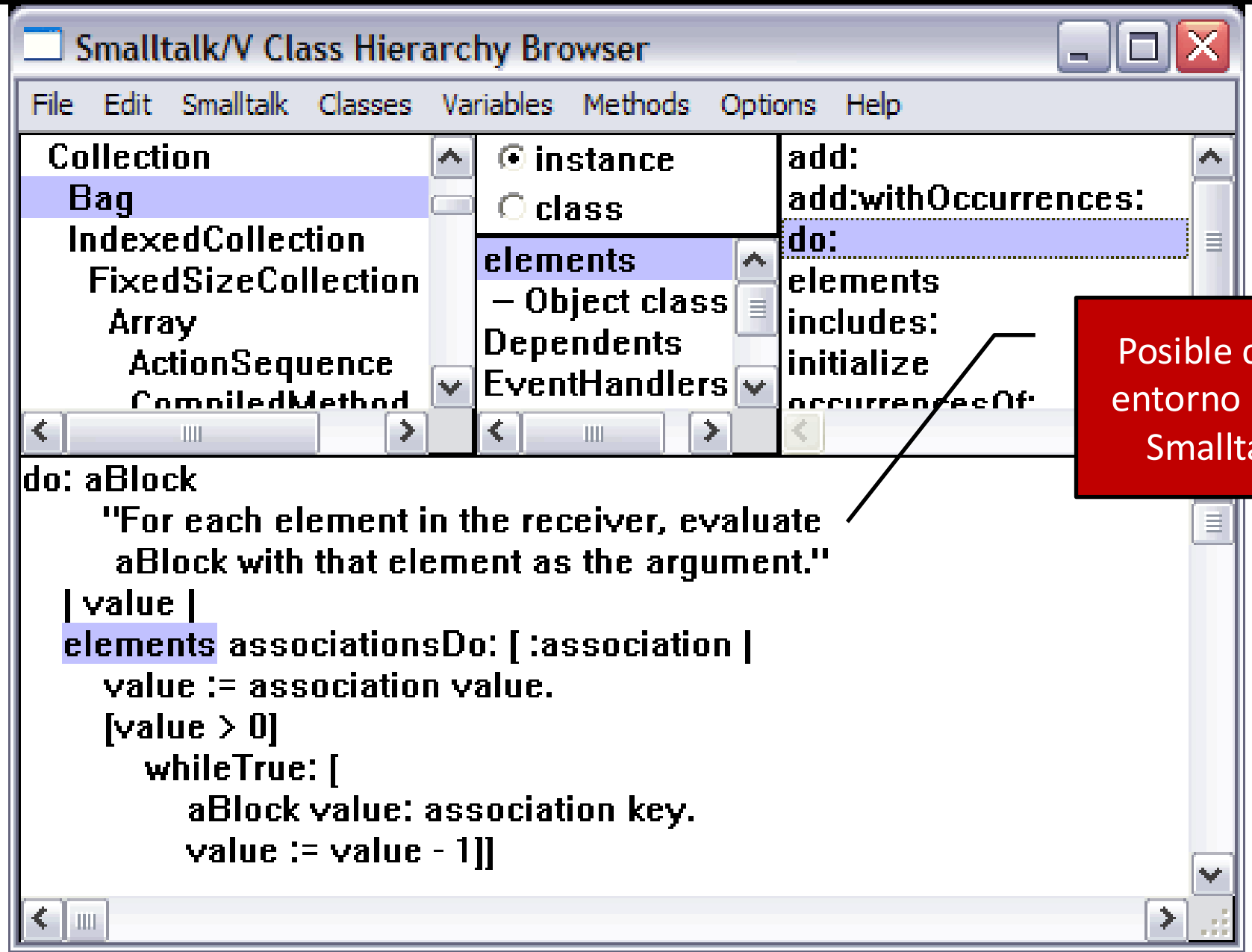
```
result := colCuentas select: [:cuenta| cuenta getSaldo > 1000]

colCuentas do:[:cuenta| cuenta reintegro:200]
```



Métodos `do:` y `select:` en clase `Collection` tenían como parámetro un único argumento que era de **tipo bloque de código** (*closures, lambda expressions*)

Sensación de libertad al programar



Posible cambiar el propio entorno implementado es Smalltalk y disponible

Contenidos

- Clases anidadas
  - Anidadas estáticas e Internas
- Programación Funcional:
  - Closures (function object)
  - **Expresiones Lambda en Java 8**
  - Iteradores en Java 8 : Streams
    - Patrón Iterador: Iteradores externos e internos
  - Tipo Optional en Java 8
- Metaprogramación e Introspección: Reflexión en Java

Java 8. Motivación

- Java no proporcionaba ***bloques* o *closures*** a diferencia de otros lenguajes OO.
- Proporcionar construcciones para facilitar un estilo de programación funcional con la POO.
- Proporcionar mecanismos para escribir código que tome ventaja de CPU multicore para procesar colecciones.



Expresiones Lambda y Streams

Java. Ordenar colecciones

- La clase `Collections` ofrece el método *static* `sort` para ordenar listas de acuerdo al orden establecido por un objeto comparador:

```
public static <T>  void sort(List<T> list, Comparator<T> c)
```

- Necesidad de crear una subclase de `Comparator<T>` para implementar el criterio de orden.

```
interface Comparator<T> {
        int compare(T o1, T o2);
}
```

- Ejercicio: Posibilidad de usar introspección para crear una clase genérica que implemente `Comparator<T>`

Java. Ordenar colecciones

```
public class Usuario {...
    private String nombre;
    private Date nacimiento;

    public String getNombre() {
        return nombre;
    }
    public int getEdad() {
        // Propiedad calculada ...
    }
}
```

```
public class ComparadorUsuariosPorEdad
        implements Comparator<Usuario> {
    @Override
    public int compare(Usuario o1, Usuario o2) {
        return o1.getEdad() - o2.getEdad();
    }
}
```

```
List<usuario> usuarios;
…
Collections.sort(usuarios, new ComparadorUsuariosPorEdad());
```

Ordenar colecciones con clases anónimas

- Evitan proliferación de clases pequeñas.
- Alternativa a no ofrecer expresiones lambda.

```
Collections.sort(
    usuarios,
    new Comparator<Usuario>() {
         @Override
         public int compare(Usuario o1, Usuario o2) {
              return o1.getNombre().compareTo(o2.getNombre());
         }
    }
);
```

Java 8. Primer ejemplo de expresión lambda

```
Collections.sort(usuarios,
    (Usuario u1, Usuario u2) -> {
        return u2.getNombre().compareTo(u1.getNombre());
    }
);
```

- Se puede simplificar la expresión dado que el compilador realiza **inferencia de tipos** y el bloque de código es una única sentencia.

```
Collections.sort(usuarios,
        (u1, u2) -> u2.getNombre().compareTo(u1.getNombre());
);
```

- Las expresiones lambda tienen acceso a las variables (locales y parámetros) y campos del contexto del código.

Formas de expresar una lambda

1.
```
Runnable process = () -> System.out.println("Hello World");
```
Sin argumentos

Bloque de código de una línea

2.
```
ActionListener al =
        event -> System.out.println("button clicked");
```
1 arg sin tipo

Bloque de código de una línea

3.
```
Runnable process = () -> {
                System.out.print("Hello");
                System.out.println(" World");
};
```
Sin argumentos

Bloque de código de 2 líneas

4.
```
BinaryOperator<Long> add = (x, y) -> x + y;
BinaryOperator<Long> addExplicit = (Long x, Long y) -> x + y;
```
2 argumentos con tipo

Bloque de 1 línea

Interfaz funcional

- Una interfaz funcional es una interfaz con un **único método** que es utilizada como tipo de una expresión lambda (no necesario en un lenguaje tipado dinámicamente).
- Una expresión lambda es un “objeto” que es un bloque de código (función sin nombre) compatible con una interfaz funcional.

```
@FunctionalInterface
interface Comparator<T> {
        int compare(T o1, T o2);
}
```

La declaración `@FunctionalInterface` es opcional.

Ejemplo de Interfaz funcional: `ActionListener`

Todas las interfaces con un único método son ahora *interfaces funcionales*

```
public interface ActionListener extends EventListener {
     // Invoked when an action occurs.
    public void actionPerformed(ActionEvent e);

}
```

```
JLabel resultado = new JLabel();

JButton botonSaludo = new JButton(“Poner título");
botonSaludo.addActionListener(ev ->
                        resultado.setText("Hola alumnos"));
```

Antes con clases anónimas, ahora con lambdas

Interfaces funcionales de propósito general en el paquete java.util.function

```
package java.util.function;

public interface Predicate<T> {
        // comprueba la condición sobre el argumento dado
        boolean test(T t);
}
public interface Consumer<T> {
        // realiza la operación sobre el argumento dado
        void accept(T t);
}
public interface Function<T,R> {
    // retorna el valor obtenido al aplicar la función sobre el argumento dado
        R apply(T t);
}
public interface Supplier<T> {
    // retorna un resultado de tipo T, una alternativa al patrón método factoría
        T get();
}
```

Interfaces funcionales de propósito general en el paquete `java.util.function`

- Representa una función que comprueba si un objeto satisface una condición.

```
interface Predicate<T> {
        boolean test(T obj);
}
```

- Representa operaciones que dado un único argumento realizan una acción y no retornan nada.

```
interface Consumer<T> {
    void accept(T obj);
}
```

- Representa un suministrador de resultados (objetos)

```
interface Supplier<T> {
    T get();
}
```

- Representa una función que acepta un argumento y devuelve un resultado.

```
interface Function<T, R> {
    R apply(T obj);
}
```

Ejemplo: Aplicar filtro concreto sobre una colección

```
public class FiltroCuentasSaldoMayor{
  public static List<Cuenta> filtrarCuentasPorSaldo(List<Cuenta> lista,int cant){
        ArrayList<Cuenta> result = new ArrayList<Cuenta>();
        //filtrar cuentas con saldo mayor que una cantidad

        for(Cuenta cu:lista)
                if (cu.getSaldo()>cant)
                                result.add(cu);
        return result;
  }

  public static void main(String[] args) {
        Cuenta c1 = new Cuenta (10000, "123", "JGM");
        Cuenta c2 = new Cuenta (200, "345", "IGM");
        Cuenta c3 = new Cuenta (3000, "567", "EGM");
        Cuenta c4 = new Cuenta (300, "890", "CMT");
        Cuenta c5 = new Cuenta (2000, "111", "JLGM");
        ArrayList<Cuenta> lista = new ArrayList<Cuenta>();
        lista.add(c1); lista.add(c2); lista.add(c3); lista.add(c4); lista.add(c5);

        for(Cuenta cu:FiltroCuentasSaldoMayor.filtrarCuentasPorSaldo(lista,1000))
                System.out.println(cu.getCcc());
        }
  }
```

Aplicar filtro con una interface

```
public class FiltroCuentas{
   public static List<Cuenta> filtrarCuentas(List<Cuenta> lista, ITest filtro){
          //cualquier filtro para listas de Cuentas
          ArrayList<Cuenta> result = new ArrayList<Cuenta>();
          for(Cuenta cu:lista)
                   if (filtro.test(cu)) result.add(cu);
          return result;
   }
   public static void main(String[] args) {…
      TestSaldoCuenta filtro = new TestSaldoCuenta(1000);
      for(Cuenta cu:FiltroCuentas.filtrarCuentas(lista, filtro))
             System.out.println(cu.getCcc());
   }
}
```

```
public interface ITest<T> {
   boolean test (T o);
}
```

```
public class TestSaldoCuenta implements ITest<Cuenta> {
    private int cantidad;
    TestSaldoCuenta(int cant) {
          this.cantidad = cant;
    }
    public boolean test(Cuenta c) {
          return c.getSaldo() > cantidad;
    }
}
```

Filtros son clases que implementan una interfaz `Itest`

Aplicar filtro con interface + clases anónima

```
public class FiltroCuentas{
   public static List<Cuenta> filtrarCuentas2( List<Cuenta> lista, ITest filtro){
         ArrayList<Cuenta> result = new ArrayList<Cuenta>();
         //cualquier filtro para listas de Cuentas
         for(Cuenta cu:lista)
                  if (filtro.test(cu))
                           result.add(cu);
         return result;
   }
   public static void main(String[] args) {…
         for(Cuenta cu:FiltroCuentas.filtrarCuentas2(
                  lista,
                  new ITest<Cuenta>(){
                           @Override
                           public Boolean test(Cuenta c) {
                                    return c.getSaldo()>1000;
                           }
                  }))
                           System.out.println(cu.getCcc());
   }
}
```

Filtros con interfaz y clases anónimas

Aplicar filtro con Predicate<T>

```
public class FiltroCuentas{
   public static List<Cuenta> filtrarCuentas3(
                        List<Cuenta> lista, Predicate<Cuenta> filtro){
        ArrayList<Cuenta> result = new ArrayList<Cuenta>();
        for(Cuenta cu:lista)
                if (filtro.test(cu))
                        result.add(cu);
        return result;
   }
   public static void main(String[] args) {…
        for(Cuenta cu:FiltroCuentas.filtrarCuentas3(
                lista, (Cuenta c) -> c.getSaldo()>1000))
            System.out.println(cu.getCcc());
   }
}
```

Realizar una acción sobre objetos de una colección que cumplen una condición

```
public class IteradorCuentas{

   public static void hacerSi(List<Cuenta> lista.
                              Predicate<Cuenta> filtro,
                              Consumer<Cuenta> action) {
         for(Cuenta cu:lista)
                  if (filtro.test(cu))
                           action.accept(cu);
   }

   public static void main(String[] args) {
         ArrayList<Cuenta> result = new ArrayList<Cuenta>();
         IteradorCuentas.hacerSi(
                  lista,
                  (Cuenta c) -> c.getSaldo()>1000,
                  (Cuenta c) -> result.add(c));
         for (Cuenta cu:result)
                  System.out.println(cu.getCcc());
   }
}
```

Ejemplo de uso de Function<T>

```
package tds.jesus;

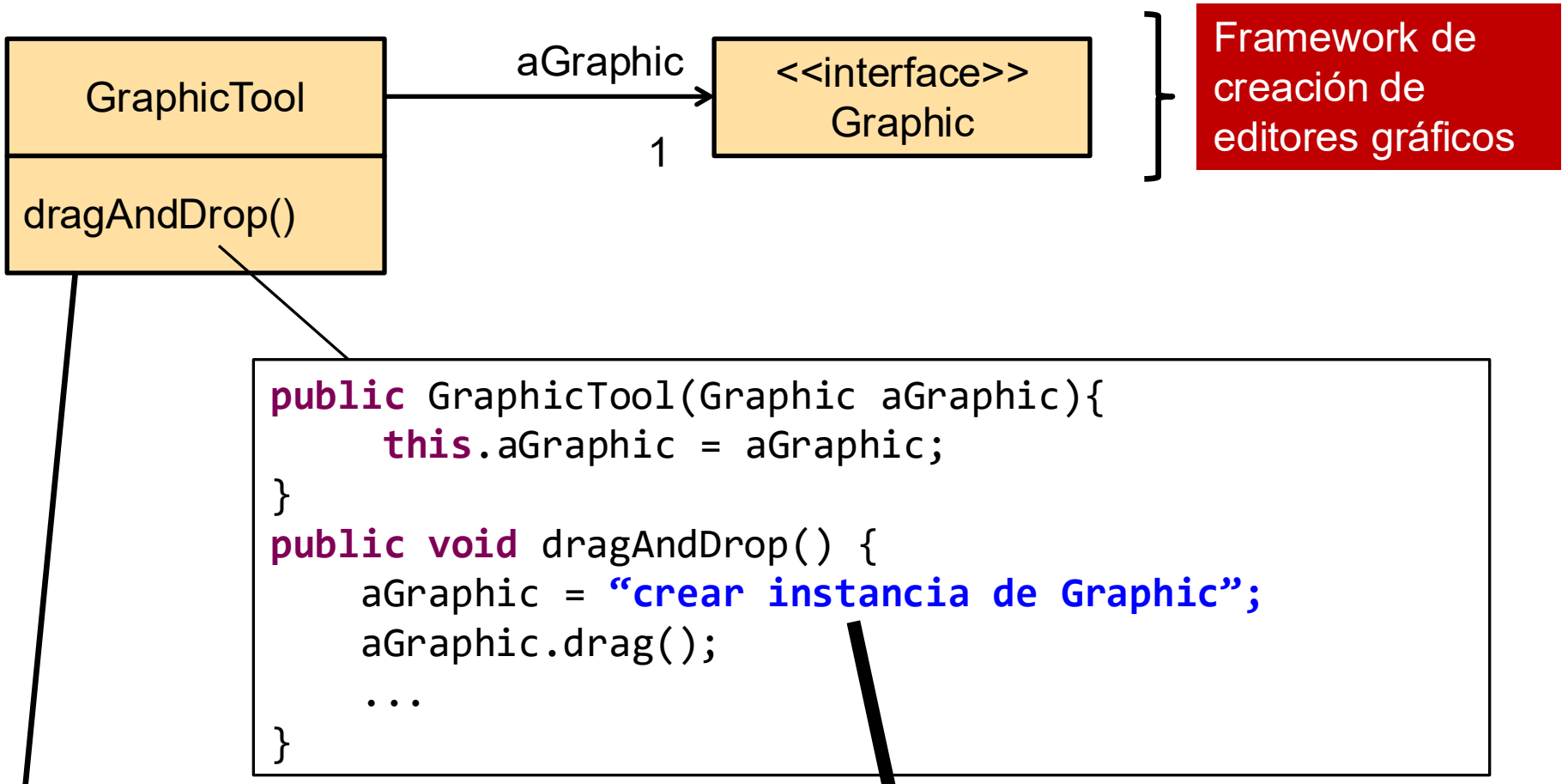
import java.util.ArrayList;
import java.util.function.Function;

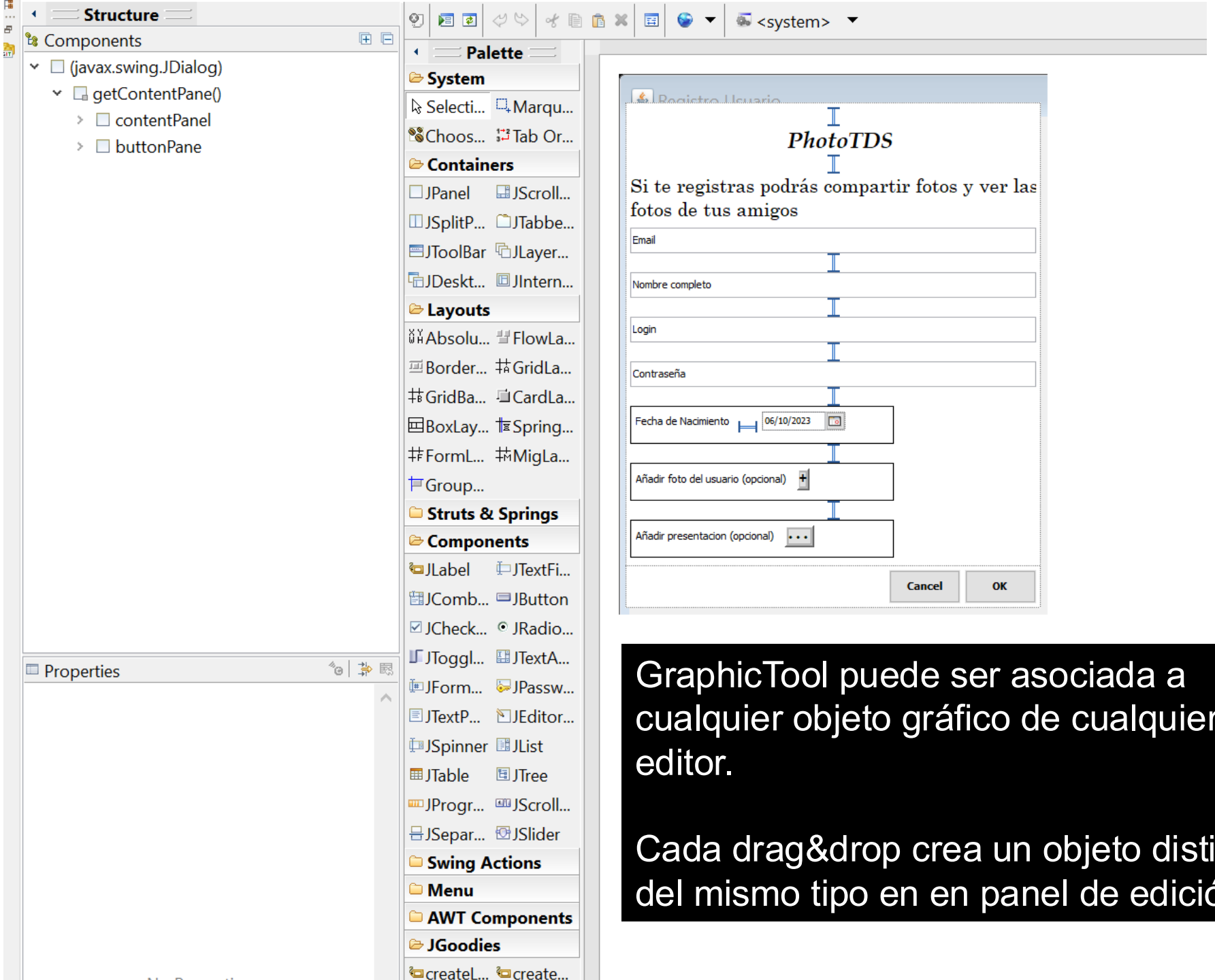
public class PruebaExpLamda {

   public static void main (String[] args){
        Cuenta c1 = new Cuenta (100, "123", "JGM");
        Cuenta c2 = new Cuenta (200, "345", "IGM");
        Cuenta c3 = new Cuenta (300, "567", "EGM");

        // asignar expresión lambda a variable
        Function<Cuenta,Integer> exp = (Cuenta c) -> c.getSaldo();
        // invocar expresión lambda
        System.out.println(exp.apply(c3));
   }
}
```

Método Factoría o Prototype con *Supplier* (revisado en tema “Patrones de diseño”)





GraphicTool puede ser asociada a cualquier objeto gráfico de cualquier editor.

Cada drag&drop crea un objeto distinto del mismo tipo en en panel de edición.

Método Factoría con *Supplier* (revisado en tema “Patrones de diseño”)

```
public class Rectangle implements Graphic {..
     private int width;
     private int height;
     private String name;
     public Rectangle(String name,
                int width, int height) {
       super();
       this.name = name;
       this.width = width;
       this.height = height;
    }
    @Override
    public String getName() {
        return name; }
    @Override
    public void drag() {..}
}
```

Método Factoría o Prototype con *Supplier* (revisado en tema “Patrones de diseño”)

```
import java.util.function.Supplier;
public class GraphicTool {
     private Supplier<Graphic> supplier;

     public void setSupplier(Supplier<Graphic> supplier) {
        this.supplier = supplier;
     }
     public void dragAndDrop() {..
        supplier.get().drag();
     }
     public void printGraphicName(){
        System.out.println(supplier.get().getName());
     }
}
public interface Graphic {
    public String getName();
}
```

Método Factoría con *Supplier* (revisado en tema “Patrones de diseño”)

```
public class App {

   public static void main(String[] args) {
        Supplier<Graphic> g = () -> new Rectangle("Caja 1",3,2);
        //se crea e inicializa un GraphicTool para rectángulos
        GraphicTool gt = new GraphicTool();
        gt.setSupplier(g);

        gt.printGraphicName();
  }
}
```

Ejemplos de `Supplier` con funciones

```
public void supplier1(){
    Supplier<Double> doubleSupplier1 = () -> Math.random();

    System.out.println(doubleSupplier1.get());
}
```

```
public void supplier2(){
       LocalDateTime ldt = LocalDateTime.now();
       System.out.println(ldt); // Muestra hora actual H1

       Supplier<LocalDateTime> dateSupplier = LocalDateTime::now;

       Thread.sleep(5000);
       System.out.println(ldt);  // Muestra hora H1
       System.out.println(dateSupplier.get()); // Muestra hora actual H2

       Thread.sleep(5000);
       System.out.println(ldt);  // Muestra hora H1
       System.out.println(dateSupplier.get()); // Muestra hora actual H3
}
```

El contexto en el que un `Supplier` resulta útil incluye, pero no se limita a:

1. **Creación de objetos**: Puedes utilizar un `Supplier` cuando quieras crear un objeto pero retrasar su construcción hasta que realmente lo necesites.

```java
Supplier<StringBuilder> builderSupplier = () -> new StringBuilder();
StringBuilder builder = builderSupplier.get();
```

2. **Operaciones Lazy**: En ciertas situaciones, es posible que desees realizar operaciones de manera perezosa (lazy), es decir, solo cuando realmente se necesiten. El `Supplier` puede ser utilizado para encapsular estas operaciones.

```java
public String expensiveOperation() {
    // Algún cálculo costoso
    return "Resultado";
}

Supplier<String> lazyOperation = this::expensiveOperation;
// No se llama a expensiveOperation hasta que se invoca lazyOperation.get()
```

4. **Factories**: Puedes usar `Supplier` para abstraer la creación de objetos, lo que puede ayudarte a implementar patrones como el Factory.

```java
public <T> T create(Supplier<T> factory) {
    return factory.get();
}

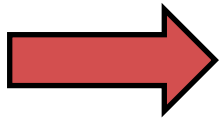
MyClass instance = create(MyClass::new);
```

Referencias a métodos

- Forma abreviada de expresar expresiones lambda que sólo invocan un método sobre el argumento.

Referencia a un método de una instancia arbitraria de una determinada clase

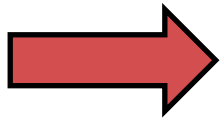
```
Function<String, Artista> f1 =
artista -> artista.getNombre();}
```



```
Function<String, Artista> f1 =
Artista::getNombre
```

Referencia a un método estático

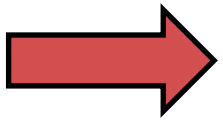
```
Function <String, Integer> f2=
cadena->Integer.parseInt(cadena);
```



```
Function <String,Integer> f2 =
Integer::parseInt
```

Referencia a un constructor

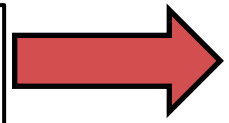
```
Supplier<Artista> s =
() -> new Artista();
```



```
Supplier<Artista> =
Artista::new
```

Referencia a un constructor con un argumento

```
Function(String, Artista) f3 =
(name)-> new Artista(name);}
```



```
Function(String, Artista) f3 =
Artista::new
```

Contenidos

- Clases anidadas
  - Anidadas estáticas e Internas
- Programación Funcional:
  - Closures (function object)
  - Expresiones Lambda en Java 8
  - **Iteradores en Java 8 : Streams**
    - **Patrón Iterador: Iteradores externos e internos**
  - Tipo Optional en Java 8
- Metaprogramación e Introspección: Reflexión en Java

Streams

- Un ***stream*** (`java.util.Stream`) es una **secuencia** de elementos que soporta operaciones secuenciales y paralelas.
- Las **colecciones** incluyen dos métodos para obtener *streams*: `stream()` y `parallelStream()y` la clase **`Stream`** ofrece **métodos para iterar** sobre sus elementos (*iteradores internos*).

Ejemplo: contar las cuentas con un saldo positivo en una colección `cuentas` de objetos `Cuenta`.

```
ArrayList<Cuenta> cuentas;

long contador = cuentas.stream()
      .filter(cu -> cu.getSaldo() > 0)
      .count();
```

Patrones de operaciones con colecciones

```
for (T:e col1)
     if (condicion(e))
                    col2.add(e);
```

Select / Filter

```
for (T:e col1)
     col2.add(accion(e));
```

Map

```
for (T:e col1)
          accion(e);
```

Do / for each

```
existe = false;
for (T:e col1)
   if (condicion(e)) {
        existe = true;
        break;
   }
```

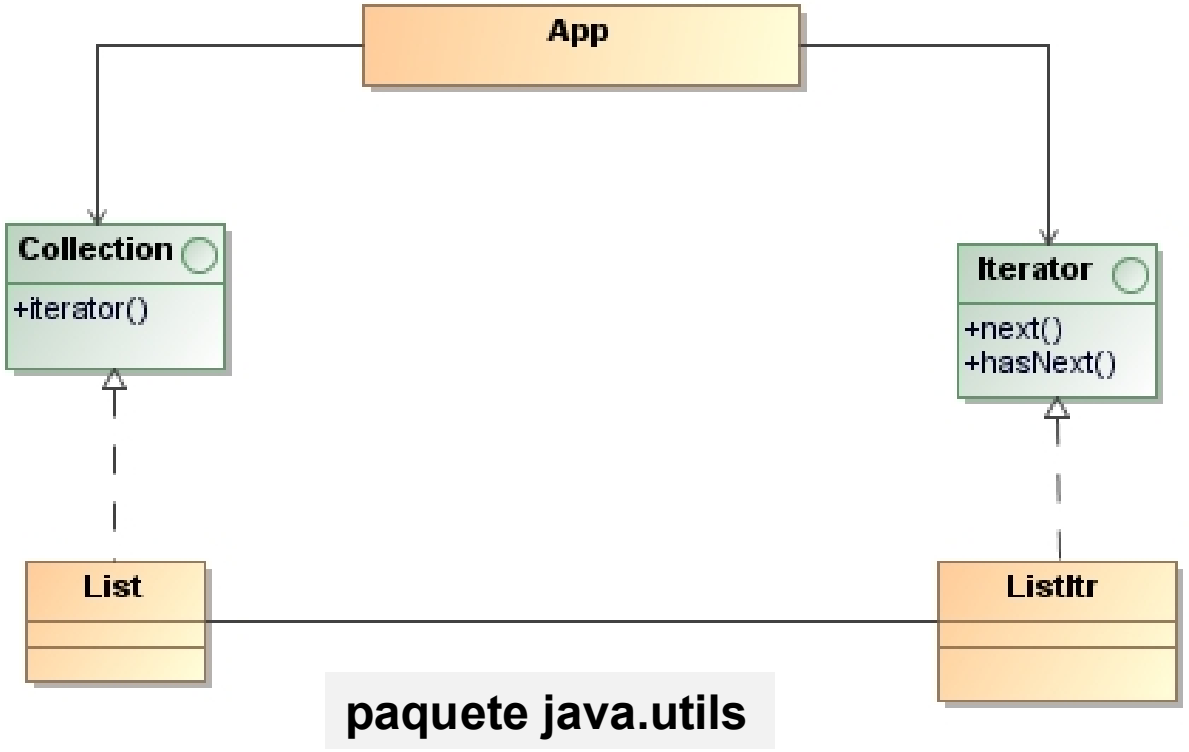
Any?

Sorting, y más

Patrón de Diseño *Iterador*

- **Propósito**
  - Proporciona una forma para acceder a los elementos de una estructura de datos sin exponer los detalles de la representación.
  - Proporcionados por mayoría de lenguajes de programación
- **Motivación**
  - Un objeto contenedor (colección) debe permitir una forma de recorrer sus elementos **sin exponer su estructura interna,** permitiendo **diferentes métodos de recorrido** y recorridos **concurrentes**
  - Las clases colección no implementan la iteración.
- **Dos posibilidades según quién controla el recorrido:**
  - Externos: el código que usa el iterador (tarea del programador)
  - **Internos**: el propio iterador (**se libera al programador que sólo dice lo que quiere**)

Iterador Externo en Java: *Iterator*




```
List<Cuenta> cuentas;

int num = 0;
Iterator<Cuenta> it = cuentas.iterator();
while (it.hasNext()) {
     Cuenta cu = it.next();
      if (cu.getSaldo>10000) num = num +1;
}
```

Iterador Interno *forEach* en Java, antes de Java 8

```
List<Cuenta> cuentas;
int num = 0;
for (Cuenta cu: cuentas)
      if (cu.getSaldo() > 10000)
                  num = num + 1;
```

Streams: Ejemplo

Dada una lista de transacciones, obtener la lista de identificadores de transacciones que son de cierto tipo (p.e., COMPRADIVISAS) ordenadas por el valor de la transacción de mayor a menor.

Java <8

```
//filtro
List<Transaction>  compradivisas = new Arraylist<>();
for(Transaction t: transactions){
      if (t.getType() == Transaction.COMPRADIVISAS)                    1
                                       compradivisas.add(t);
}
//ordenación
Collections.sort(compradivisas, new Comparator(){
      public int compare(Transaction t1, Transaction t2){              2
            return t2.getValue().compareTo(t1.getValue());
  }
});
//creación de una nueva lista
List<Integer> transactionIds = new ArrayList<>();
for(Transaction t: compradivisas){                                     3
      transactionsIds.add(t.getId());
}
```

Patrones de operaciones con colecciones

```
for (T:e col1)
     if (condicion(e))
                    col2.add(e);
```

Select / Filter

```
for (T:e col1)
     col2.add(accion(e));
```

Map

```
for (T:e col1)
          accion(e);
```

Do / for each

```
existe = false;
for (T:e col1)
   if (condicion(e)) {
        existe = true;
        break;
   }
```

Any?

Sorting, y más

Streams: Ejemplo

Dada una lista de transacciones, obtener la lista de identificadores de transacciones que son de cierto tipo (p.e., COMPRADIVISAS) ordenadas por el valor de la transacción de mayor a menor.

Java < 8

```
//filtro
List<Transaction>  compradivisas = new Arraylist<>();
for(Transaction t: transactions){
      if(t.getType() == Transaction.COMPRADIVISAS)
                                    compradivisas.add(t);
}
//ordeno
Collections.sort(compradivisas, new Comparator(){
      public int compare(Transaction t1, Transaction t2){
            return t2.getValue().compareTo(t1.getValue());
  }
});
//construyo nueva lista
List<Integer> transactionIds = new ArrayList<>();
for(Transaction t: compradivisas){
      transactionsIds.add(t.getId());
}
```

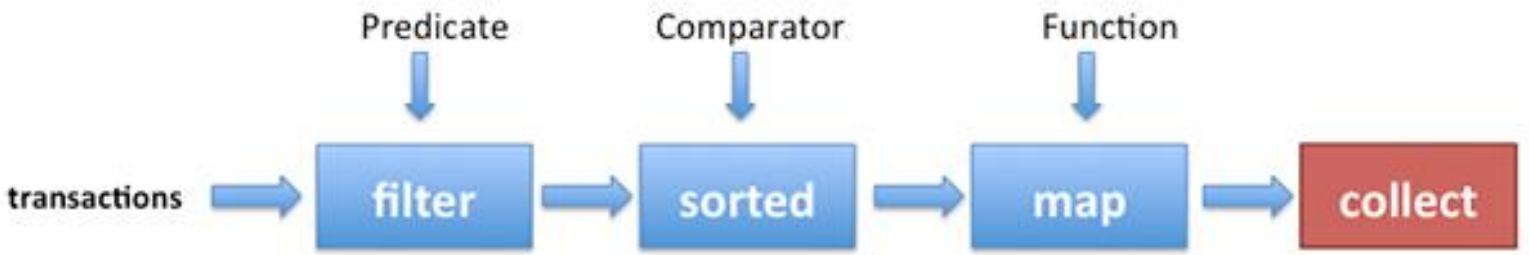
Filter

Sort

Map

Streams: Ejemplo 1

Dada una lista de transacciones, obtener la lista de identificadores de transacciones que son de cierto tipo (p.e., `COMPRADIVISAS`) ordenadas por el valor de la transacción de mayor a menor.



Java 8

```
List<Integer> transactionsIds = transactions.stream()
      .filter(t -> t.getType() == Transaction.COMPRADIVISAS)
      .sorted(comparing(Transaction::getValue).reversed())
      .map(Transaction::getId)
      .collect(toList());
```

```
static import Comparator.*
static import Collectors.*
```

(http://www.oracle.com/technetwork/articles/java/ma14-java-se-8-streams-2177646.html)

Iterador **interno: Propiedades**

- El recorrido es controlado internamente, el programador es liberado.
- Permiten iterar sobre una colección y ejecutar operaciones como “seleccionar todos los elementos que cumplen una condición” o “aplicar una operación sobre cada elemento”.
- Un iterador interno requiere **parámetros** que son operaciones: condición y/o acción **=>** Requieren el mecanismo llamado ***“bloque de código”, closure, o expresiones lambda.***
- Comunes en lenguajes OO tipados dinámicamente como Smalltalk, Python, y Ruby.

```
public static void main(String[] args) {
    List<Integer> numbers =
                Arrays.asList(5, 15, 25, 3, 8);
    Predicate<Integer> condition = n -> n > 10;

    long count = numbers.stream()
            .filter(condition)
            .count();
}
```

Java 8

```
val numbers = List(5, 15, 25, 3, 8)
val condition: Int => Boolean = n => n > 10
val count = numbers.count(condition)
```

Scala

```
numbers = [5, 15, 25, 3, 8, 12]
condition = lambda n: n > 10
count = len([n for n in numbers if
                                condition(n)])
```

Python

Operaciones más comunes sobre streams

```
import static java.util.stream.Collectors.*;
..
public static void main(String[] args) {

// collect -> convertir un stream en una colección
   List<String> collected1 = Stream.of("uno", "dos", "tres", "cuatro")
          .collect(Collectors.toList());

// map -> retorna stream resultado de aplicar una acción sobre cada elemento de un stream
   List<String> collected2 = collected1.stream()  // convierte colección en stream
          .map(s -> s.toUpperCase())
          .collect(toList());

// filter -> retorna stream de elementos que cumplen una condición
   List<String> collected3 = collected1.stream()
          .filter(s -> s.endsWith("s"))
          .collect(toList());

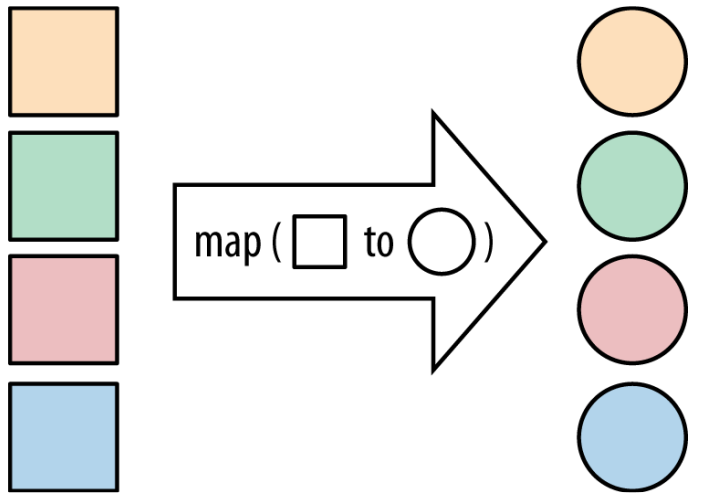
// forEach -> ejecuta una acción sobre cada elemento de la secuencia
   collected1.stream()
          .forEach(s -> System.out.println(s));

// ordena según orden natural o recibe como argumento un Comparator
   collected1.stream()
          .sorted();
```

Operaciones con Streams
(figura tomada de “Java 8 Lambdas” de R. Warburton)

- Método **`map`**:
  - Aplica una función sobre cada elemento de un stream y retorna otro stream con los elementos que resultan de aplicar la función sobre cada elemento de entrada (mismo número de elementos).
  - Acepta como parámetro una *función* (`Function<T, R>`)

```
Stream.of("uno",
"dos", "tres",
"cuatro")
```



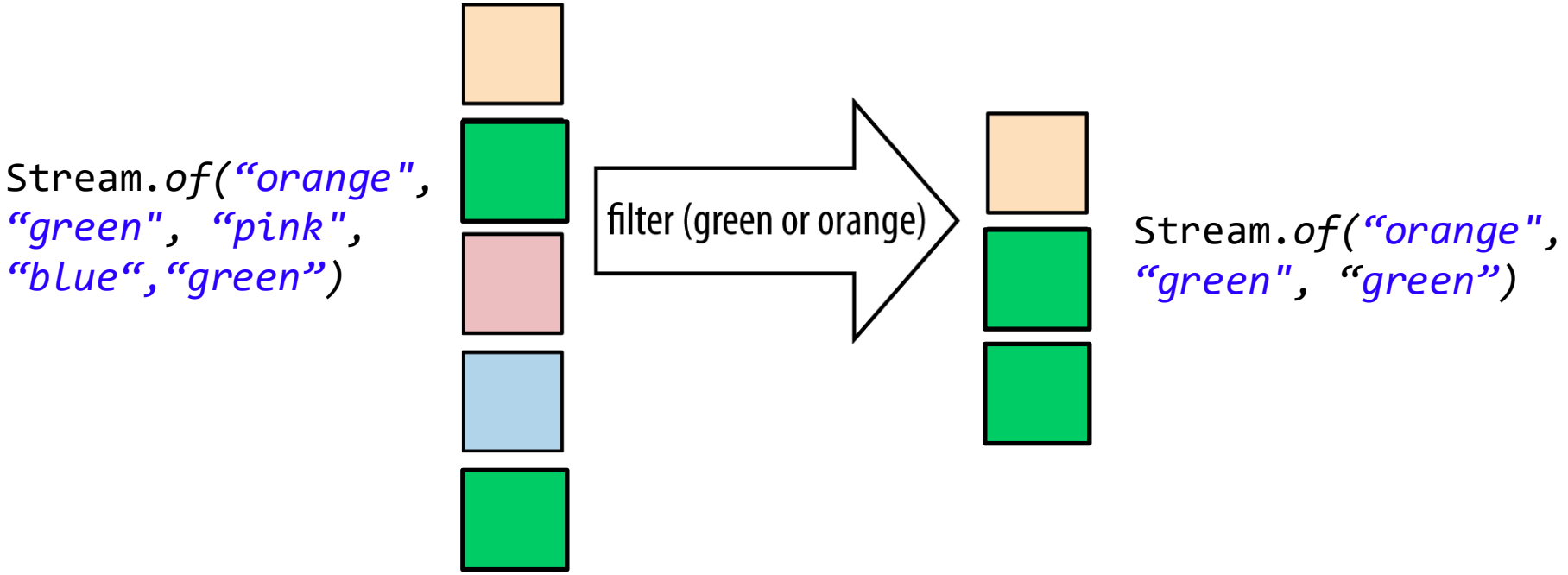

```
Stream.of(“UNO",
“DOS", “TRES",
“CUATRO")
```

```
.map(s -> s.toUpperCase());
```

Function<T, R>

Operaciones con Streams
(figura tomada de “Java 8 Lambdas” de R. Warburton)

- Método **`filter`**:
  - Retorna un stream con los elementos del stream original que **cumplen una condición.**
  - Acepta como parámetro un *predicado* (`Predicate<T>`




```
.filter(s->s.equals("green") || s.equals("orange"))
```

Predicate<T>

Operaciones con Streams

- **`forEach`**:
  - **Operación terminal** que aplica una acción sobre cada elemento.
  - Acepta como parámetro un *consumidor* (`Consumer<T>`)
- **`anyMatch`**, **`noMatch`** y **`allMatch`**:
  - **Operaciones terminales** que retornan un booleano indicando si se cumple un predicado en alguno (`any`), ninguno (`no`) o todos (`all`) los elementos.
  - Acepta como parámetro un *predicado* (`Predicate<T>`)
- **`sorted`**:
  - Ordena el stream según un criterio de ordenación.
  - Acepta como parámetro un *comparador* (`Comparator<T>`).
  - Retorna un *stream* con la secuencia ordenada.
  - Tiene una versión sobrecargada sin parámetros que aplica el orden natural de los elementos.
- **`distinct:`**
  - Elimina duplicados manteniendo el orden.

Operaciones con streams

- **`count()`**:
  - **Operación terminal** que retorna el número de objetos de un stream como un `long`.
- **`limit(n)`**:
  - Selecciona los primeros n elementos de un stream.
- **`collect()`**:
  - **Operación terminal** que transforma el resultado del procesamiento del stream en una colección.
  - Acepta como parámetros métodos que transforman *streams* en colecciones, p.e. listas (`Collectors.toList()`), sets (`Collectors.toSet()`, o usar `Collectors.toCollection(..)`.

```
Stream<String> stream = Stream.of("c", "a", "b");
TreeSet<String> result = stream.collect(
                                    Collectors.toCollection(TreeSet::new));
```

Operaciones Terminales vs. No Terminales

Operaciones **no terminales.**

- Retornan un nuevo Stream, lo que permite encadenar múltiples operaciones.
- No modifican el Stream original, sino que generan uno nuevo.
- Son perezosas, no se ejecutan hasta que se invoca una operación terminal.
- Ejemplos: filter, map, sorted, limit, flatmap

Operaciones **terminales.**

- Producen un resultado no-stream, por lo tanto, finalizan el procesamiento del stream.
- Desencadenan la ejecución de las operaciones intermedia no terminales.
- Ejemplos: count, anyMatch, forEach, findFirst

Operaciones con streams: Eficiencia

- Para mejorar la eficiencia, las operaciones sobre stream pueden ejecutarse inmediatamente (eager) o ser postergadas (lazy).
- Las operaciones intermedias (no terminales) son **lazy**, mientras las operaciones *terminales* son **eager**.
- Ejemplos de beneficios:
  - Cuando se aplica una operación `limit()`, no tratar todos los elementos
  - Aplicar operaciones en un solo recorrido del stream.

```
Stream<String> stream = Stream.of("a", "ab", "abc", "abcd", "abcde“);

stream.filter(s -> {
    System.out.println("Filtrando: " + s);
    return s.length() > 2;
})
.map(s -> {
    System.out.println("Mapeando: " + s);
    return s.toUpperCase();
})
.limit(2)
.collect(Collectors.toList());
```

(Ejemplo proporcionado por ChatGPT)

Operaciones con streams: Eficiencia

```
Stream<String> stream = Stream.of("a", "ab", "abc", "abcd", "abcde“);

stream.filter(s -> {
    System.out.println("Filtrando: " + s);
    return s.length() > 2;
})
.map(s -> {
    System.out.println("Mapeando: " + s);
    return s.toUpperCase();
})
.limit(2)
.collect(Collectors.toList());
```

```
Filtrando: a
Filtrando: ab
Filtrando: abc
Mapeando: abc
Filtrando: abcd
Mapeando: abcd
```

Aunque el stream tiene 5 elementos, el filtrado y el mapeo sólo se aplica hasta encontrar 2 elementos que cumplan con el criterio debido a la operación `limit(2)`.

(Ejemplo proporcionado por ChatGPT)

Collectors

- Proporciona métodos estáticos que retornan instancias de clases que implementan `Collector` y realizan **operaciones de reducción** sobre streams:

```
// Recolectar una lista
    List<String> listaNombresPersonas = personas.stream()
              .map(Persona::getNombre)
              .collect(Collectors.toList());

// Recolectar un set
    Set<String> setNombresPersonas = personas.stream()
            .map(Persona::getNombre)
            .collect(Collectors.toSet());

// Recolectar un map
    Map<String, String> mapDNINombrePersona = personas.stream()
        .collect(Collectors.toMap(Persona::getDNI, Persona::getNombre));

    Map<String, Persona> mapDNIPersona = personas.stream()
        .collect(Collectors.toMap(Persona::getDNI, Function.identity());
```

Collectors

- Proporciona métodos estáticos que retornas instancias de clases que implementan `Collector` y realizan **operaciones de reducción** sobre streams:

```
// Recolectar a una colección específica
     Set<String> set = personas.stream()
            .map(Persona::getNombre)
            .collect(Collectors.toCollection(TreeSet::new));

// Estadísticas -> Calcular una suma de valores
     int total = empleados.stream()
            .collect(Collectors.summingInt(Empleado::getSalario));

// Agrupación -> ej.: agrupar empleados por departamento
     Map<Departmento, List<Empleado>> empPorDepto = empleados.stream()
         .collect(Collectors.groupingBy(Empleado::getDepartamento));
```

Streams. Ejemplos para `usuarios: ArrayList<Usuario>`

Obtener una lista con sus nombres:

```
List<String> nombres = usuarios.stream()
        .map(u -> u.getNombre())
        .collect(Collectors.toList());
```

Mostrar en consola los nombres de los usuarios en orden alfabético:

```
usuarios.stream()
        .map(u -> u.getNombre())
        .sorted()
        .forEach(System.out::println);
```

Comprobar si algún usuario tiene más de 18 años:

```
boolean hayAdulto = usuarios.stream()
            .anyMatch(u -> u.getEdad() > 18);
```

Obtener los nombres de los usuarios mayores de 18 años.

```
Set<String> nomAdultos = usuarios.stream()
        .filter(u -> u.getEdad() > 18)
        .map(u -> u.getNombre())
        .collect(Collectors.toSet());
```

Streams. Ejemplos para `cuentas: ArrayList<Cuenta>`

Dada una **lista de cuentas**, aplicar un reintegro de 1000 euros a cuentas con saldo > de 1000 euros

```
cuentas.stream()
      .filter(cu->cu.getSaldo()>1000)
      .forEach(cu -> cu.reintegro(1000));
```

Obtener la lista ordenada por nombre de los titulares de cuentas con un saldo mayor a 1000 euros.

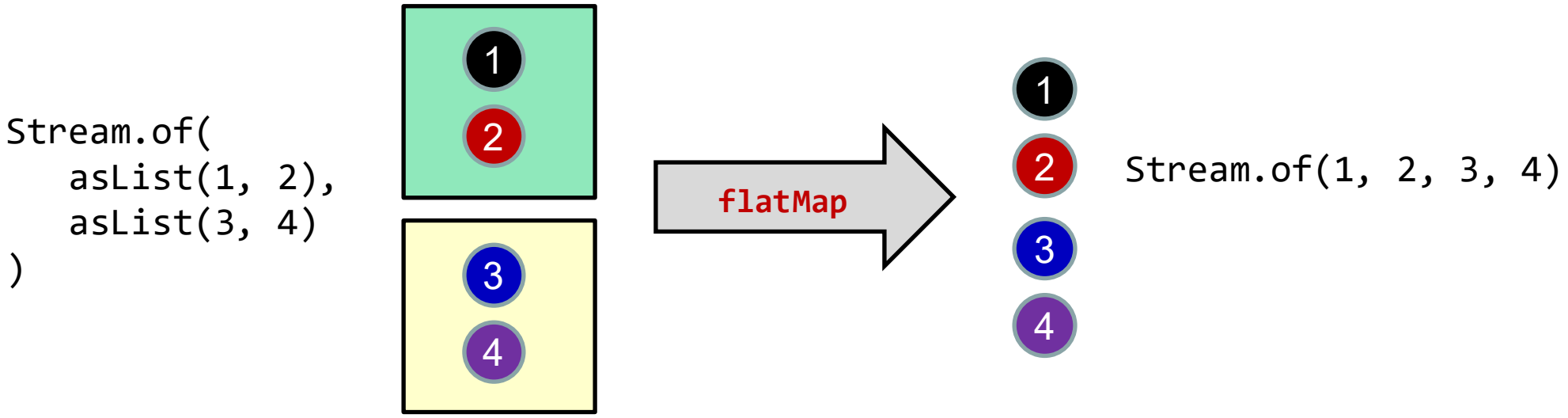
```
List<String> titularesMil = cuentas.stream()
                  .filter(cu->cu.getSaldo()>1000)
                  .map(cu ->cu.getTitular())
                  .sorted()
                  .collect(toList());
```

Operación `flatMap`

- **`flatMap`**
  - Convertir un stream de colecciones en un único stream con todos los elementos concatenados (operación ***flatten***). Ejemplo:

```
List<Integer> resultado = Stream.of(asList(1, 2), asList(3, 4))
        .flatMap(listanum -> listanum.stream())
        .collect(toList());

assertEquals(asList(1, 2, 3, 4), resultado);
```



`flatMap``(listanum -> listanum.stream())`

Operaciones `min` y `max`

- **`max` y `min`**
  - Encontrar el mayor o menor elemento en un stream que se ordena con un objeto `Comparator<T>`.
  - Retornan un `Optional<>` (veremos más adelante)

Ejemplos:

```
ArrayList<Cancion> canciones;
Cancion cancionMasCorta = canciones.stream()
        .min(Comparator.comparing(Cancion::getDuracion()))
        .get();
```

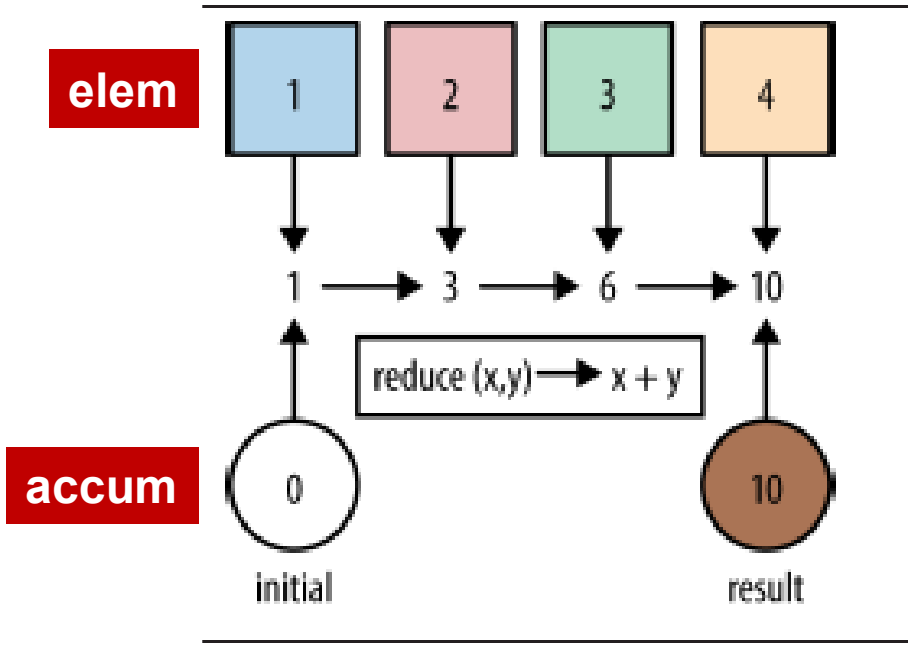
```
ArrayList<Persona> personas;
Optional<Persona> personaMayor = personas.stream()
         .max(Comparator.comparing(Persona::getEdad));
```

Operación `reduce`

- **`reduce`**

```
T reduce (T identity, BinaryOperator<T> accumulator)
```

  - Genera un resultado a partir de una secuencia de elementos.
  - `count()`, `summingInt`, `max` y `min()` son ejemplos de operaciones *reduce*.
  - Se suele combinar con la operación `map` (p.e., *MapReduce* en Hadoop).




```
int count = Stream.of(1, 2, 3, 4)
    .reduce(0, (accum, elem)
                -> accum + elem);
```

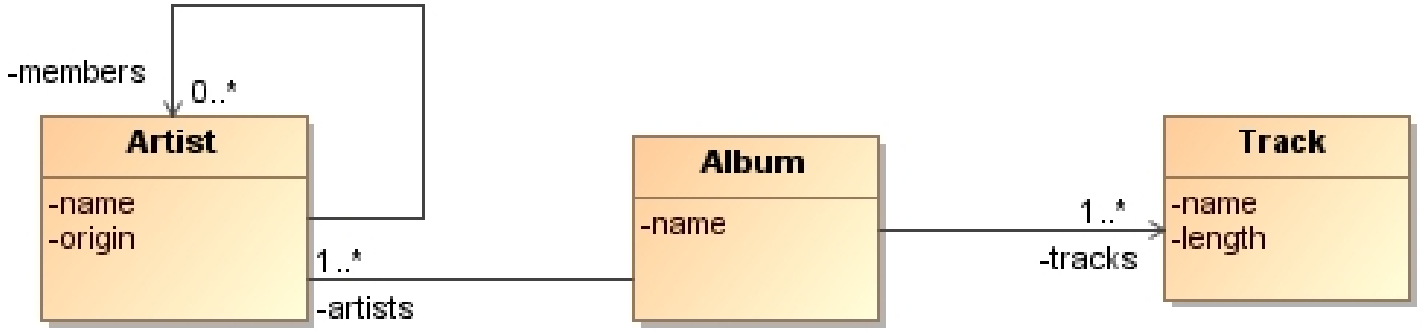
`summingtInt()`

Ejemplos **map + reduce**

```
List<String> nombres = Arrays.asList("Ana", "Juan", "Pedro", "Lucía");
// Convertir los nombres a mayúsculas y concatenarlos String
nombresConcatenados = nombres.stream()
      .map(String::toUpperCase)
      .reduce("", (a, b) -> a + b + " ");
```

```
List<Integer> numeros = Arrays.asList(1, 2, 3, 4, 5);
// Suma de los cuadrados
int sumaCuadrados = numeros.stream()
      .map(n -> n * n)
      .reduce(0, Integer::sum);
```

Ejercicio: Refactoring en "*Java 8 Lambdas*" de R. Warburton



En una lista de álbumes, encontrar todos los nombres de canciones (`Track`*) con una duración mayor de un minuto.*

```
public Set<String> findLongTracks(List<Album> albums) {
        Set<String> trackNames = new HashSet<>();
        for(Album album : albums) {
                for (Track track : album.getTrackList()) {
                        if (track.getLength() > 60) {
                                String name = track.getName();
                                trackNames.add(name);
                        }
                }
        }
        return trackNames;
}
```

Ejercicio: Refactoring en "*Java 8 Lambdas*" de R. Warburton

En una lista de álbumes, encontrar todos los nombres de canciones (`Track`*) con una duración mayor de un minuto.*

```
public Set<String> findLongTracks(List<Album> albums) {
        Set<String> trackNames = new HashSet<>();
        albums.stream()
                .forEach(album -> {
                        album.getTracks().stream()
                            .filter(track -> track.getLength() > 60)
                            .map(track -> track.getName())
                            .forEach(name -> trackNames.add(name));
                });
        return trackNames;
}
```

¿Puede conseguirse un código más compacto?

Ejercicio: Refactoring en "*Java 8 Lambdas*" de R. Warburton

En una lista de álbumes, encontrar todos los nombres de canciones (`Track`*) con una duración mayor de un minuto.*

```
public Set<String> findLongTracks(List<Album> albums) {
        Set<String> trackNames = new HashSet<>();

        albums.stream()
           .flatMap(album -> album.getTracks().stream())
           .filter(track -> track.getLength() > 60)
           .map(track -> track.getName())
           .forEach(name -> trackNames.add(name));

        return trackNames;
}
```

¿Se puede evitar la variable local `trackNames`?

Ejercicio: Refactoring en "*Java 8 Lambdas*" de R. Warburton

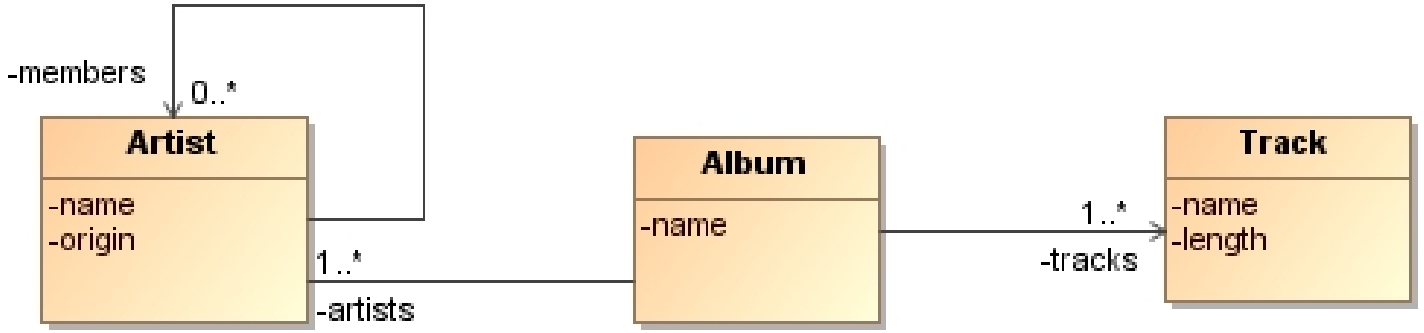
En una lista de álbumes, encontrar todos los nombres de canciones (Track) con una duración mayor de un minuto.

```
public Set<String> findLongTracks(List<Album> albums) {

        return albums.stream()
                .flatMap(album -> album.getTracks().stream())
                .filter(track -> track.getLength() > 60)
                .map(track -> track.getName())
                .collect(toSet());
}
```

Ejemplo de programación functional en Java 8

Ejercicios: Refactoring en *Java 8 Lamdas* de R. Warburton



En una lista de álbumes, encontrar todos los nombres de canciones (`Track`*) con una duración mayor de un minuto.*

```
public class MusicServiceController{
    public Set<String> findLongTracks(List<Album> albums) {
        Set<String> trackNames = new HashSet<>();
        for(Album album : albums)
            for (Track track : album.getTrackList())
                if (track.getLength() > 60) {
                    String name = track.getName();
                    trackNames.add(name);
                }
        return trackNames;
    }
}
```

¡¡ Se viola patrón Experto !!

Ejercicios: Refactoring en *Java 8 Lamdas* de R. Warburton

```
public class MusicServiceController{
   public Set<String> findLongTracks(List<Album> albums) {
       return albums.stream()
              .flatMap(album -> album.getTracks().stream())
              .filter(track -> track.getLength() > 60)
              .map(track -> track.getName())
              .collect(toSet());
}
```

¿Se viola el Experto?

¿Puede considerarse un código más compacto y se evita dispersión?

Refactoring anterior pero sin violar GRASP

```java
public class Track {..
   String name;
   int length;

   public String getTitle() {
        return name;
   }
   public int getLength() {
        return length;
   }
   public boolean isLong(int length) {
       return this.length > length;
   }
}
```

```java
public class Album { ..
   String title;
   List<Track> tracks;

   public Album(String title) {
     super();
     this.title = title;
     this.tracks = new ArrayList<Track>();
   }
   public List<Track> getTracks() {
      return tracks; }
   public void addTrack(Track t) {
      tracks.add(t);
   }
   public List<Track> getLongTracks(int length) {
       return tracks.stream()
           .filter(t -> t.isLong(length))
           .collect(Collectors.toList());
   }
}
```

Ejercicios: Refactoring en *Java 8 Lamdas* de R. Warburton

```
public class MusicServiceController{
   public Set<String> findLongTracks(List<Album> albums) {
       return albums.stream()
              .flatMap(album -> album.getLongTracks(60).stream())
              .map(track -> track.getName())
              .collect(toSet());
}
```

Refactoring anterior pero sin violar GRASP

```
public class Artist {..
   List<Album> albums;
   String name;

   public Artist(String name) {
     super();
     this.name = name;
     this.albums = new ArrayList<Album>();
   }
   public void add(Album a) {
          albums.add(a);
    }
    public String getName() {
          return name;
    }
    public Set<Track> getLongTracks (int length) {
       return albums.stream()
                  .flatMap (a -> a.getLongTracks(length).stream())
                  .collect(Collectors.toSet());
    }
```

Refactoring anterior pero sin violar GRASP

```
public class Artist {..
   List<Album> albums;
   String name;

   public Artist(String name) {
     super();
     this.name = name;
     this.albums = new ArrayList<Album>();
   }
   public void add(Album a) {
          albums.add(a);
    }
    public String getName() {
          return name;
    }
    public Set<Track> getLongTracks (int length) {
       return albums.stream()
                  .flatMap (album -> album.getTracks().stream())
                  .filter(track ->track.isLong(length())
                  .collect(Collectors.toSet());
    }
```

Programación funcional con lambdas y streams

- Código más simple y conciso.
- Se expresa **qué** transformación se realiza en vez del **cómo** se realiza la transformación.
- Código funcional que expresa la **intención** del programador, en vez de una descripción de cómo hacerlo.
- Facilitan escribir código libre de efectos laterales.
  - Se manejan variables que no pueden ser modificadas (actúan como finales). No hay asignaciones a variables.

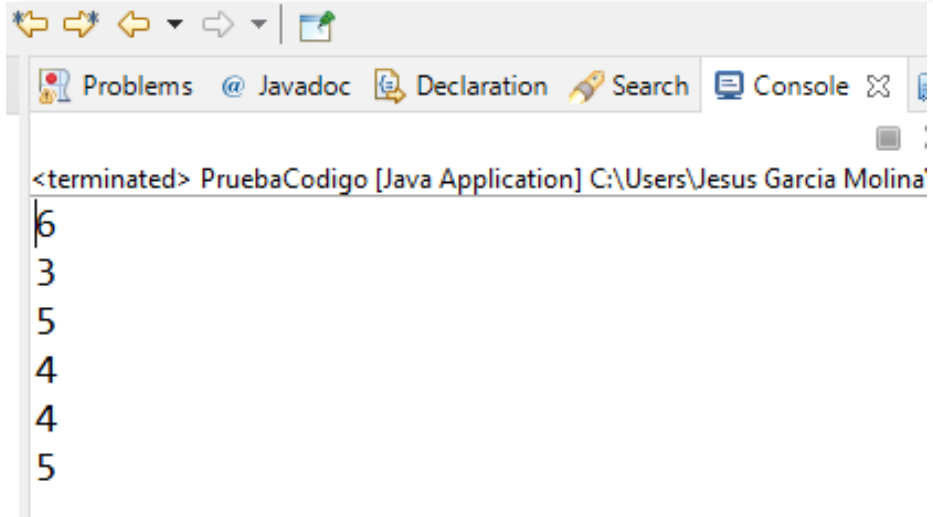
Ejercicio de Examen POO, 2017

Utilizando el procesamiento basado en *streams*, implementa un método `transformar` que reciba como parámetro una colección de strings y una *función* a aplicar sobre los objetos de dicha colección. El resultado de la ejecución será una nueva colección de string que contiene los objetos obtenidos al aplicar la función. Como ejemplo de uso, declara e inicializa una lista de cadenas y aplica el método `transformar` utilizando esa lista como parámetro con el fin de obtener una colección de esas cadenas en letras mayúsculas.

```
public class PruebaCodigo {
    public static <String,R> List<R> transformar(Collection<String> col,
                                                  Function<String,R> function) {
        return col.stream()
                .map(function)
                .collect(Collectors.toList());
    }
    public static void main(String[] args) {
         List<String> als = Arrays.asList("orange","red","green",
                                          "pink", "blue","green");
         Function<String, String> f = String::toUpperCase;
         PruebaCodigo.transformar(als, f).stream()
                                 //forEach(s -> System.out.println(s));
                                 .forEach(System.out::println);
         }
}
```

Ejercicio Examen POO, Junio 2017

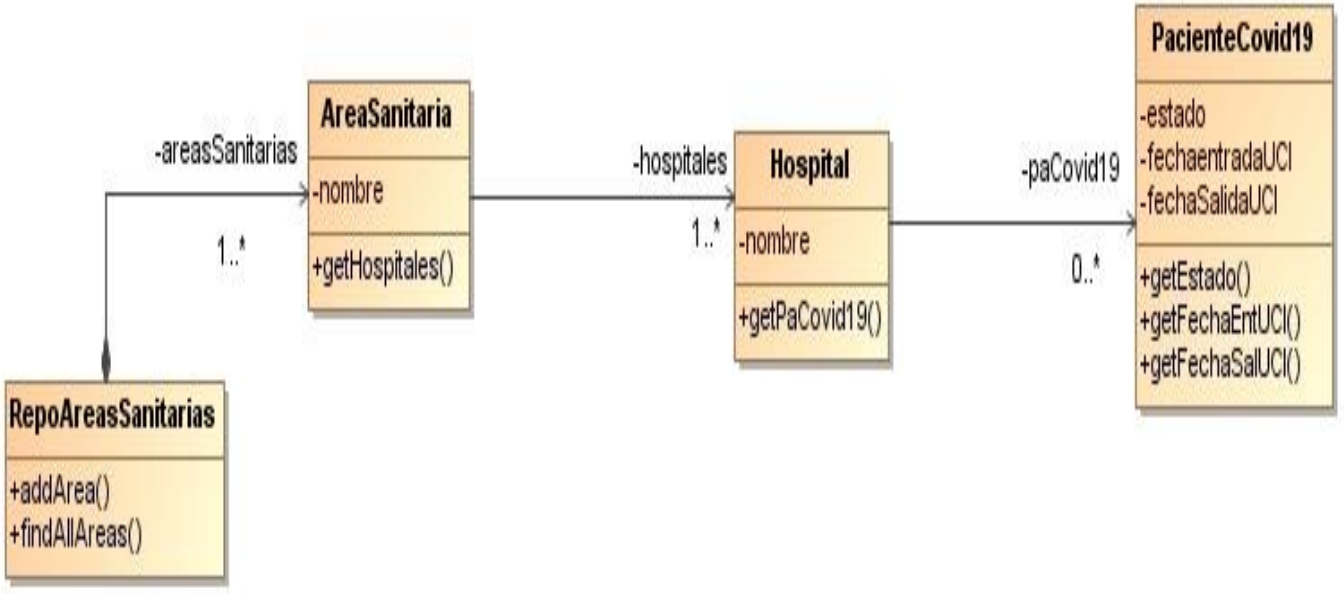
```
public class PruebaCodigo {
    public static <String,R> List<R> transformar(Collection<String> col,
                                                 Function<String,R> function) {
        return col.stream()
                .map(function)
                .collect(Collectors.toList());
    }
    public static void main(String[] args) {
         List<String> als = Arrays.asList("orange","red","green",
                                          "pink", "blue","green");
         PruebaCodigo.transformar(als,String::length).stream()
                                 //forEach(s -> System.out.println(s));
                                 .forEach(System.out::println);
         }
}
```



Se obtienen las longitudes de las cadenas en vez de pasar a mayúsculas

Ejercicio Examen TDS, Junio 2020

Sea una aplicación de gestión de la pandemia Covid-19 en una región que tiene varias áreas de salud, y cada hospital pertenece a un área. El diagrama de clases de abajo muestra las principales clases del dominio de esa aplicación. Cada objeto de la clase `AreaSanitaria` registra una lista de objetos `Hospital`, y éstos una lista de `PacienteCovid19`. `PacienteCovid19` tiene, entre otros, un campo `estado` que es una lista de enumerados que registra los estados de un paciente Covid-19 (INGRESO, UCI, PLANTA, ALTA), y los campos `fechaEntradaUCI` y `fechaSalidaUCI` que registran la fecha de entrada y salida de la UCI (se ignora que un paciente podría ingresar varias veces en UCI hasta su alta). `RepoAreasSanitarias` es un catálogo que almacena las instancias de `AreaSanitaria` y está implementado como un singleton, su método `findAllAreas()` retorna todas las áreas sanitarias.



Ejercicio Examen TDS, Junio 2020

Dado el diagrama de clases anterior, reescribe el código de abajo en Java 8 utilizando los métodos de la clase `Stream`. Este código calcula **el número total de pacientes que han estado en UCI en un intervalo de fechas.** Reescribe dicho código en forma del siguiente método: `public long findTotalPacsUCI (LocalDate fe, LocalDate fs)`. El parámetro `fe` corresponde a la fecha de entrada a UCI y `fs` a la de salida.

```
long count = 0;
for (AreaSanitaria area:
    RepoAreasSanitarias.INSTANCE.findAllAreas())
    for (Hospital ho: area.getHospitales())
        for (PacienteCovid19 pac: ho.getPacientesCovid())
            if ((pac.getEstado().includes(Estado.UCI)) &&
               (fe.isBefore(pac.getFechaAltaUCI()) ||
                fe.isEqual(pac.getFechaAltaUCI())) &&
               (fs.isAfter(pac.getFechaBajaUCI()) ||
                fs.isEqual(pac.getFechaBajaUCI())))
                                                count = count+1;
```

Ejercicio Examen TDS, Junio 2020

```
public long findTotalPacsUCI(LocalDate fe, LocalDate fs) {
   return RepoAreasSanitarias.INSTANCE.findAllAreas().stream()
                  .flatMap(area -> area.getHospitales().stream())
                  .flatMap(hos -> hos.getPacientesCovid().stream())
                  .filter(pac -> pac.isUCI(fe,fs))
                  .count();
}
```

Contenidos

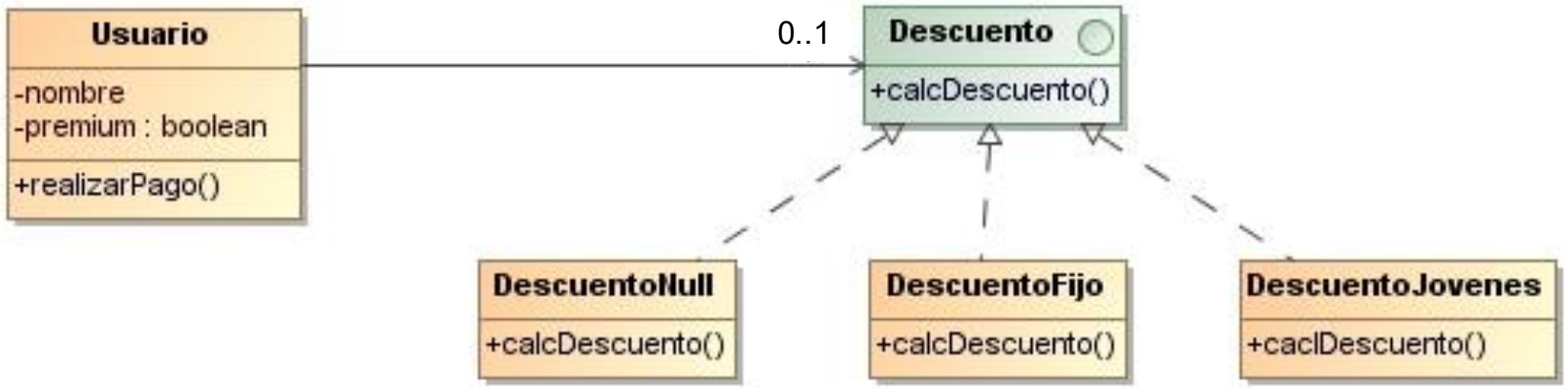
- Clases anidadas
  - Anidadas estáticas e Internas
- Programación Funcional:
  - Closures (function object)
  - Expresiones Lambda en Java 8
  - Iteradores en Java 8 : Streams
    - Patrón Iterador: Iteradores externos e internos
  - **Tipo Optional en Java 8**
- Metaprogramación e Introspección: Reflexión en Java

Problemas con `Null`

- El valor **`null`** o **nil** existe en la mayoría de los lenguajes para representar **la falta de un valor** (the *absence of a value*) desde que se introdujo en Algol W en 1965.
  - Su creador Tony Hoare se refiere a él como “my billion dolar mistake”.
- **Problemas** en el uso de `null`:
  - Fuente de error, excepción `NullPointerException` en Java
  - Código menos legible por las comprobaciones.
  - No tiene significado semántico y puede ser asignado a una variable de cualquier tipo referencia (agujero en el sistema de tipos)
  - Rompe la filosofía de lenguajes OO tipados estáticamente de ocultar los punteros (*pointer*).
- **Soluciones**:
  - Patrón *Null Object*
  - *Safe Navigation operator* en Groovy o tipo `Option[T]` *en Scala.*

Patrón “Null Object”

- Alternativa al uso de `null` para indicar la ausencia de un objeto.
- Evita comprobar si una variable es null antes de una llamada.

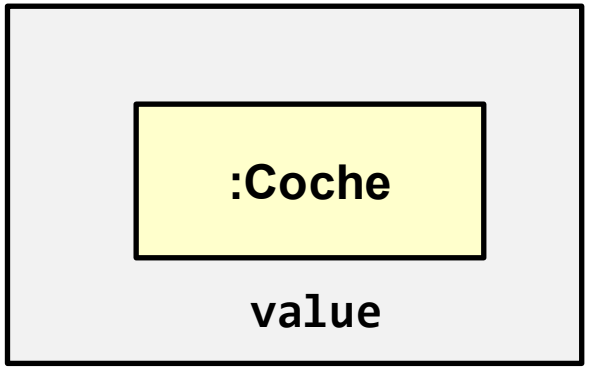



```
class DescuentoNull implements Descuento {..
   // no hay descuento
   public double  calcDescuento() {
         return 0;
   }
}
```

Optional

- Alternativa al uso de `null`, forma explícita de representar y tratar que un valor puede estar ausente.
- Facilita comprobar si una variable referencia o no a un objeto.
- La clase `Optional<T>` representa que una variable de tipo T puede referenciar o no a un objeto.
- Un `optional` es un contenedor que puede estar en uno de dos estados posibles.

```
Optional<Coche> coche;
```



`Optional<Coche>` **con un valor**



`Optional<Coche>` **sin valor**

empty

https://www.oracle.com/technical-resources/articles/java/java8-optional.html

Clase `Optional`: Métodos de creación

```
public final class Optional<T> {..
    private static final Optional<?> EMPTY = new Optional<>();
    private final T value;

    public static <T> Optional<T> empty() {
        Optional<T> t = (Optional<T>) EMPTY;
        return t;
    }

    public static <T> Optional<T> of(T value) {
        return new Optional<>(value);
    }

    public static <T> Optional<T> ofNullable(T value) {
        return value == null ? empty() : of(value);
    }
```

Crea un Optional vacío

Crea un Optional con un valor

Crea un Optional con un valor que puede ser nulo

Clase Optional: Métodos de consulta

```
public final class Optional<T> {..
```

```
public T get() {
    if (value == null)
        throw new NoSuchElementException("No value present");
    return value;
}
```

No aconsejable para obtener un valor

```
public boolean isPresent() {
                return value != null;
}
public boolean isEmpty() {
                return value == null;
}
```

Comprobar si tiene un valor distinto de null o null, mejor usar **ifPresent(action)**

```
public T orElse(T other) {
        return value != null ? value : other;
}
```

Retorna el valor distinto de null, si no tiene "other"

```
public void ifPresent(Consumer<? super T> action) {
        if (value != null)
                action.accept(value);
}
```

Ejecuta la acción si tiene un valor. También ifPresentOrElse

Clase Optional: Ejemplo findFirst()

```
public class ServicioUsuario {
    public Optional<Usuario> buscarPorId (String id) {
        return usuarios.stream()
            .filter(usuario -> usuario.getId().equals(id))
            .findFirst();
    }
}
```

```
public class App {
   public static void main(String[] args) {
       ServicioUsuario servicio = new ServicioUsuario(); // Caso: U
       Optional<Usuario> resultado = servicio.buscarPorId("1");
       //Caso: Usuario encontrado
       resultado.ifPresent(usuario -> System.out.println("Usuario encontrado:"
                                                + usuario.getNombre()));
       //Caso: Usuario no encontrado
       resultado = servicio.buscarPorId("99");
       resultado.ifPresentOrElse(
            usuario -> System.out.println("Usuario encontrado: " + usuario.getNombre()),
            () -> System.out.println("Usuario no encontrado.")
       );
    }
}
```

Clase `Optional`: Ejemplo `findAny()`

- `findAny()` (clase `Stream`) retorna un `Optional` con un elemento del stream o un `Optional` empty si el stream está vacío.
- Operación no determinista proporcionada para procesamiento paralelo.

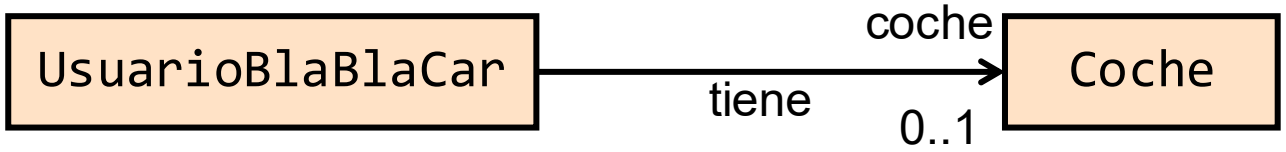
```
Optional<Usuario> usuario = usuarios.stream()
        .filter(u -> “JesusGM".equals(u.getName()))
        .findAny();
```

```
usuario.ifPresent(usuario -> System.out.println(usuario));
```

Recomendaciones:

- No utilices `Optional` como argumento de un método. Está diseñado principalmente para ser un tipo de retorno.
- No utilices `get()` sin antes comprobar si el `Optional` tiene un valor.
- Siempre es más seguro usar métodos como `orElse`, `orElseGet` o `ifPresent`.

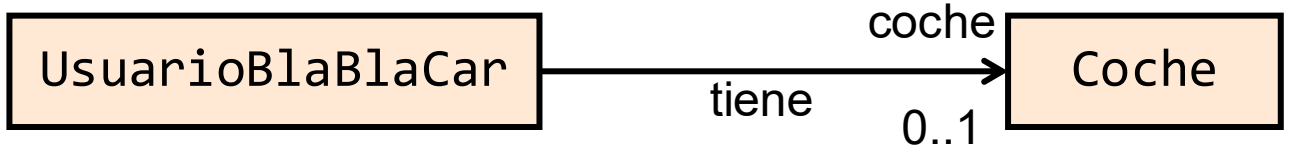
Optional: Ejemplo




```
public final class UsuarioBlablacar {
    private Optional<Coche> coche;  //no serializable
    private String nombre;

    public UsuarioBlablacar (String nombre,Coche coche) {
        this.nombre = nombre;
        this.coche = Optional.ofNullable(coche)
    }
    public UsuarioBlablacar (String nombre) {
        this.nombre = nombre;
        this.coche = Optional.empty();
    }
    public Optional<Coche> getCoche()  {
         return coche;
    }
}
```

Optional: Ejemplo




```
public final class Controlador {..
    private UsuarioBlablacar usuarioActual;

    public void crearViaje(..) {..
        if (usuarioActual.getCoche().isPresent()) {
                Viaje viaje = new Viaje(..)
}
```

También `ifPresent()`

O mejor definimos `tieneCoche()` en `UsuarioBlablacar`

```
public boolean tieneCoche() {
      return coche.isPresent) {
}
```

Entonces ..

```
public void crearViaje(..) {..
      if (usuarioActual.tieneCoche()) {
              Viaje viaje = new Viaje(..)
}
```

El tipo `Optional<T>` en Java ofrece varias ventajas en comparación con el uso directo del valor `null`. Estas ventajas están enfocadas en escribir código más claro, seguro y expresivo. Aquí te presento algunas de las ventajas principales:

1. **Intencionalidad explícita**: Al usar `Optional<T>`, estás declarando explícitamente en tu API que un valor puede estar ausente. Esto hace que el contrato sea más claro para los desarrolladores que utilizan esa API.
2. **Reduce el riesgo de `NullPointerException`**: Uno de los errores más comunes en Java es el `NullPointerException` (NPE). Al usar `Optional`, estás tratando la ausencia de un valor como un caso que debe ser manejado explícitamente, lo que puede reducir la aparición de NPEs.
3. **Operaciones en cadena**: Con `Optional`, puedes realizar operaciones en cadena (como `map`, `filter`, `ifPresent`) sin tener que hacer comprobaciones de `null` en cada paso. Esto lleva a un código más limpio y legible.
4. **Elimina la ambigüedad**: En algunas APIs, un valor `null` puede tener múltiples significados: que no hay valor, que ocurrió un error, que el valor no ha sido inicializado aún, etc. Usar `Optional` elimina esta ambigüedad al representar claramente la ausencia de un valor.
5. **Facilita patrones funcionales**: `Optional` se integra bien con el paradigma de programación funcional introducido en Java 8, permitiendo que se utilice en combinación con expresiones lambda y streams.
6. **Mejora la semántica**: Métodos que devuelven `Optional` en lugar de `null` ofrecen una mejor semántica. Por ejemplo, considera un método que busca un valor y puede no encontrarlo. Devolver un `Optional.empty()` es más semántico que simplemente devolver `null`.

7. **Evita comprobaciones innecesarias**: Con el uso de `Optional`, puedes evitar comprobaciones innecesarias de `null`. En su lugar, puedes usar métodos como `ifPresent` o `orElse` para manejar valores presentes o ausentes.
8. **Desalienta el uso de `null`**: El uso de `null` ha sido fuente de numerosos problemas y debates en la comunidad de desarrollo. Al proporcionar una alternativa en la forma de `Optional`, Java desalienta el uso de `null` como una "solución" para representar la ausencia de valores.

A pesar de estas ventajas, es importante notar que `Optional` no está destinado a reemplazar completamente el valor `null` en Java ni a ser usado en todos los contextos. De hecho, hay situaciones en las que el uso excesivo de `Optional` puede complicar el código en lugar de simplificarlo. Es esencial comprender cuándo y cómo usarlo adecuadamente para aprovechar sus beneficios.

Sin Optional

Sin `Optional`, tendríamos que hacer múltiples comprobaciones `null` para evitar `NullPointerExceptions`:

java Copy code

```java
public String obtenerEmail(Usuario usuario) {
    if (usuario != null) {
        Cuenta cuenta = usuario.getCuenta();
        if (cuenta != null) {
            Perfil perfil = cuenta.getPerfil();
            if (perfil != null) {
                return perfil.getEmail();
            }
        }
    }
    return "Email no disponible";
}
```

Con Optional

Usando `Optional`, podemos aprovechar el estilo funcional para simplificar este proceso:

java Copy code

```java
public String obtenerEmailConOptional(Usuario usuario) {
    return Optional.ofNullable(usuario)
                   .map(Usuario::getCuenta)
                   .map(Cuenta::getPerfil)
                   .map(Perfil::getEmail)
                   .orElse("Email no disponible");
}
```



En el diseño de una clase, cuándo un campo debería ser de tipo Optional.

Se recomienda

Pero …

considerar …

El uso de `Optional<T>` para campos de una clase en Java es un tema debatido en la comunidad. Sin embargo, hay ciertas recomendaciones y prácticas que puedes considerar:

1. **No usar `Optional<T>` para campos**: A menudo se recomienda no usar `Optional` como tipo de campo en una clase. En cambio, `Optional` se sugiere más para su uso en las APIs especialmente para valores que podrían no estar presentes, como el valor de retorno de un método.
2. **Evitar el uso de `null` para `Optional<T>`**: Si decides usar `Optional` como campo, nunca debes asignarle un valor `null`. Esto iría en contra del propósito principal de `Optional`, que es evitar `null`. En lugar de `null`, usarías `Optional.empty()`.
3. **Uso en constructores y setters**: Si decides utilizar `Optional` para un campo, asegúrate de que los constructores y setters tomen el tipo real (por ejemplo, `String` en lugar de `Optional<String>`) y luego conviertan ese valor en un `Optional` dentro del método.
4. **Acceso al campo `Optional`**: Al acceder al campo, puedes devolverlo como un `Optional` para que los consumidores puedan usarlo de manera funcional, aprovechando los métodos que `Optional` ofrece.
5. **Pensar en la semántica**: Si la ausencia de un valor tiene un significado específico en tu dominio, entonces usar `Optional` puede tener sentido. Por ejemplo, si estás modelando un sistema donde la ausencia de una fecha de vencimiento significa que algo nunca vence, entonces un `Optional<LocalDate>` para la fecha de vencimiento podría ser adecuado.
6. **Evitar la sobrecarga**: Si bien `Optional` es una herramienta valiosa, también introduce un sobrecarga en términos de memoria y complejidad. A menudo, un simple valor `null` y una documentación adecuada podrían ser suficientes.
7. **Considere la serialización**: Si estás serializando objetos (por ejemplo, a JSON), ten en cuenta cómo las bibliotecas de serialización manejan `Optional`. Algunas bibliotecas pueden tener problemas o requerir configuración adicional.

En resumen, mientras que `Optional` es una herramienta útil para modelar la ausencia de valores en las APIs y evitar `NullPointerExceptions`, su uso como tipo de campo en las clases debe considerarse cuidadosamente. Es más valioso y comúnmente aceptado tipo de retorno de métodos o en parámetros de método donde se quiere indicar

Re

ChatGPT

La recomendación de no utilizar `Optional` como tipo para los campos en las clases de Java, y en su lugar usarlo principalmente en los métodos de retorno, se basa en varias razones relacionadas con las mejores prácticas de diseño de software y la intención original detrás de `Optional`:

1. **Intención Semántica de `Optional`**: `Optional` fue introducido en Java 8 como una forma de expresar claramente la ausencia potencial de un valor de retorno. Su uso principal es proporcionar una alternativa más segura y expresiva a los valores `null`, evitando así `NullPointerExceptions`. No fue diseñado para ser usado como un tipo de campo de clase.
2. **Uso de Recursos Innecesario**: Usar `Optional` como tipo de campo puede llevar a un uso innecesario de memoria y a una sobrecarga de rendimiento. Cada `Optional` es un objeto adicional, y su uso como campo en cada objeto de una clase puede resultar en una cantidad significativa de objetos `Optional` adicionales, aumentando la carga sobre el recolector de basura.
3. **No Serializable**: `Optional` no es `Serializable`, lo que puede causar problemas si tu clase necesita ser serializada, por ejemplo, para ser transmitida en una red, almacenada en una base de datos, o para uso en frameworks que requieren serialización.
4. **Comportamiento no Intuitivo en Colecciones**: Si utilizas `Optional` como tipo de campo, trabajar con colecciones de objetos (como `List` o `Set`) puede ser más complicado y menos intuitivo, especialmente si necesitas filtrar, mapear o reducir basándote en estos campos `Optional`.
5. **Complejidad Añadida en el Modelo de Datos**: Al usar `Optional` como tipo de campo, añades una capa adicional de complejidad en tu modelo de datos. Esto puede hacer que el código sea más difícil de leer y mantener, especialmente para otros desarrolladores que pueden no estar familiarizados con tu código.
6. **Problemas con Frameworks de Persistencia**: Muchos frameworks ORM (Object-Relational Mapping) y de persistencia no están diseñados para trabajar bien con

Contenidos

- Clases anidadas
  - Anidadas estáticas e Internas
- Programación Funcional:
  - Closures (function object)
  - Expresiones Lambda en Java 8
  - Iteradores en Java 8 : Streams
    - Patrón Iterador: Iteradores externos e internos
  - **Tipo Optional en Java 8**
- **Metaprogramación e Introspección: Reflexión en Java**

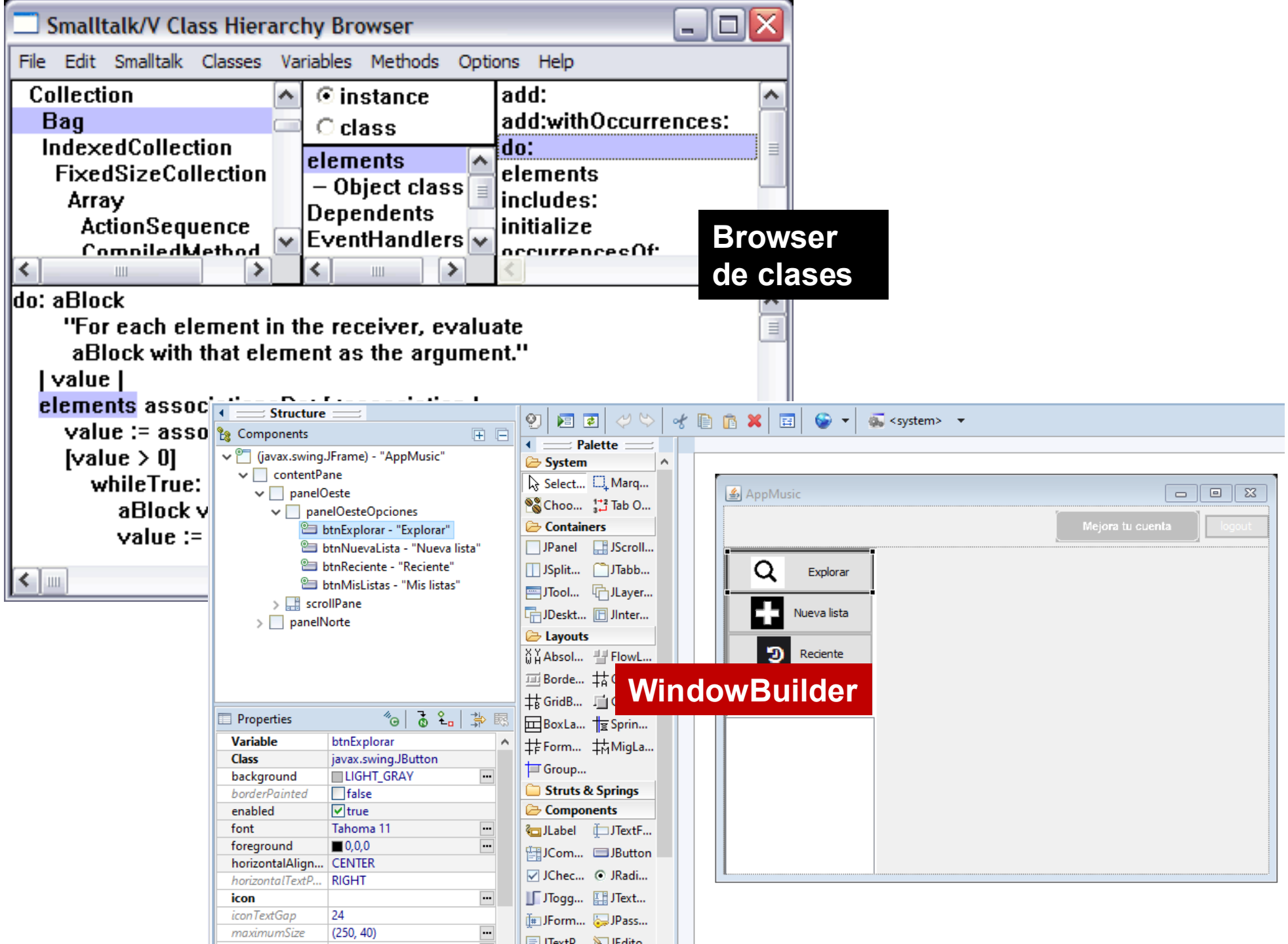
Metaprogramación e Introspección

- Una **clase puede ser considerada un objeto** para ser manipulada programáticamente --> ¿Cuál es su clase?
- Concepto de *Metaclase:* clase que describe clases, sus instancias son clases.

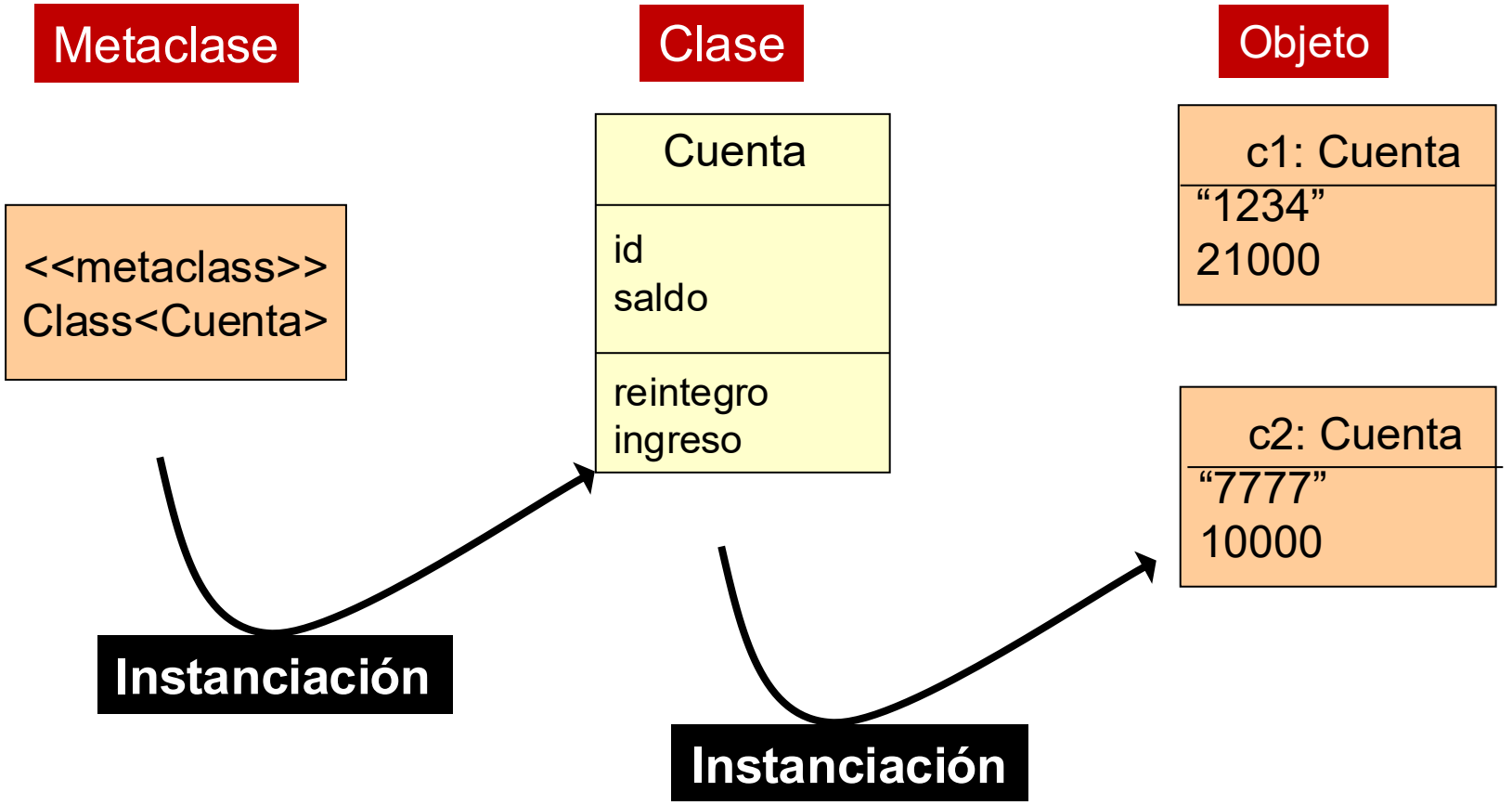
- Un lenguaje proporciona metaprogramación si ofrece la posibilidad de **manipular código** de forma programática.
  - Analizar y modificar la estructura y comportamiento de un programa, p.e. crear o modificar clases.
- Un lenguaje proporciona introspección o reflexión si ofrece la posibilidad de **analizar** código de forma programática.
- ***Smalltalk, Python* y Ruby** son lenguajes OO tipados dinámicamente que permiten metaprogramación, ***Java*** ofrece introspección, todos ellos manejan metaclases, pero de distinto modo.

Metaprogramación e Introspección

- Con metaprogramación o introspección, los **elementos software** (clases, atributos, métodos, ..) son representados por clases --> metaclases.
- Una metaclase tiene atributos que registran propiedades de una clase (p.e., campos y métodos en Java) y métodos para su acceso (en ambos casos) y modificación (metaprogramación).
- **Posibilidades como:**
  - Los argumentos de un método puede ser clases o incluso métodos.
  - Consultar los elementos de una clase, p.e., si tiene un método dado o si implementa una determinada interface (introspección)
  - Crear o modificar clases en tiempo de ejecución (metaprogramación)
  - Crear objetos sabiendo el nombre de la clase.
- **Utilidad** en programación avanzada, cuando se manejan entidades software, por ejemplo, crear inspectores de código OO, browsers de clases, builders de GUI, lenguajes específicos del dominio (DSL), ..


Smalltalk/V Class Hierarchy Browser
File Edit Smalltalk Classes Variables Methods Options Help
Collection
Bag
IndexedCollection
FixedSizeCollection
Array
ActionSequence
instance
class
elements
– Object class
Dependents
EventHandlers
add:
add:withOccurrences:
do:
elements
includes:
initialize
do: aBlock
"For each element in the receiver, evaluate
aBlock with that element as the argument."
| value |
elements
value := asso
[value > 0]
whileTrue:
aBlock v
value :=
Browser de clases
Structure
Components
(javax.swing.JFrame) - "AppMusic"
contentPane
panelOeste
panelOesteOpciones
btnExplorar - "Explorar"
btnNuevaLista - "Nueva lista"
btnReciente - "Reciente"
btnMisListas - "Mis listas"
scrollPane
panelNorte
Properties
Variable btnExplorar
Class javax.swing.JButton
background LIGHT_GRAY
borderPainted false
enabled true
font Tahoma 11
foreground 0,0,0
horizontalAlign... CENTER
horizontalTextP... RIGHT
icon
iconTextGap 24
maximumSize (250, 40)
Palette
System
Containers
Layouts
Struts & Springs
Components
<system>
AppMusic
Mejora tu cuenta
logout
Explorar
Nueva lista
Reciente
WindowBuilder

Metaclase



- ¿Cuántas metaclases hay?
  - Una para todas las metaclases con genericidad (p.e., Java)
  - Una para cada clase (p.e., Smalltalk)

Introspección o Reflexión en Java

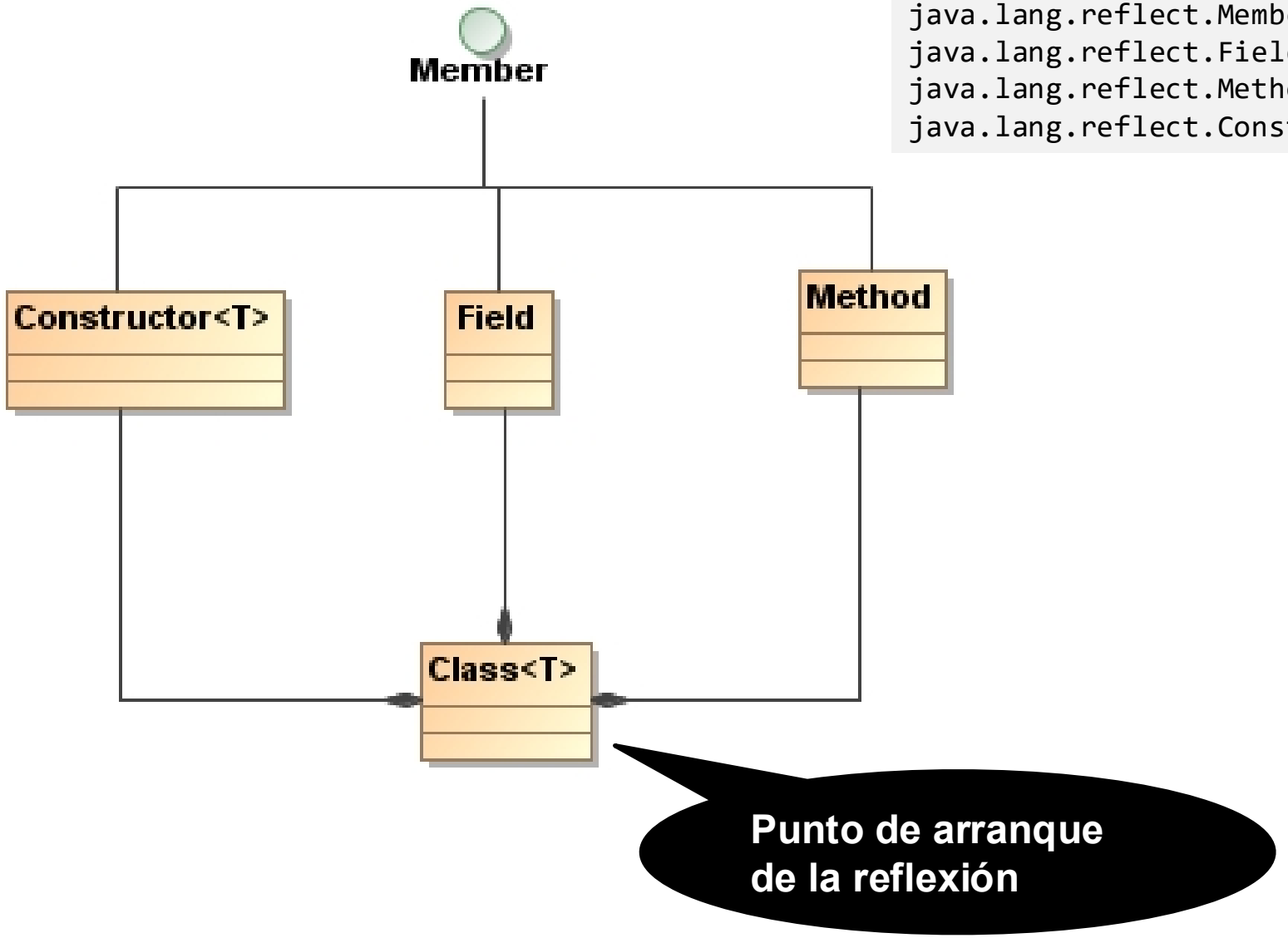
```
void metodo1 (Class<?> c) {
  “Crear un objeto a partir de una metaclase dada como argumento”
}
```

```
void metodo2 (String c) {
    “Obtener la metaclase de la clase cuyo nombre es dado por un string
    y usarla para crear un objeto de dicha clase”
}
```

```
void metodo3 (Method m, Object[] args) {
    “Invocar un método dado como argumento”
}
```

Introspección o Reflexión en Java

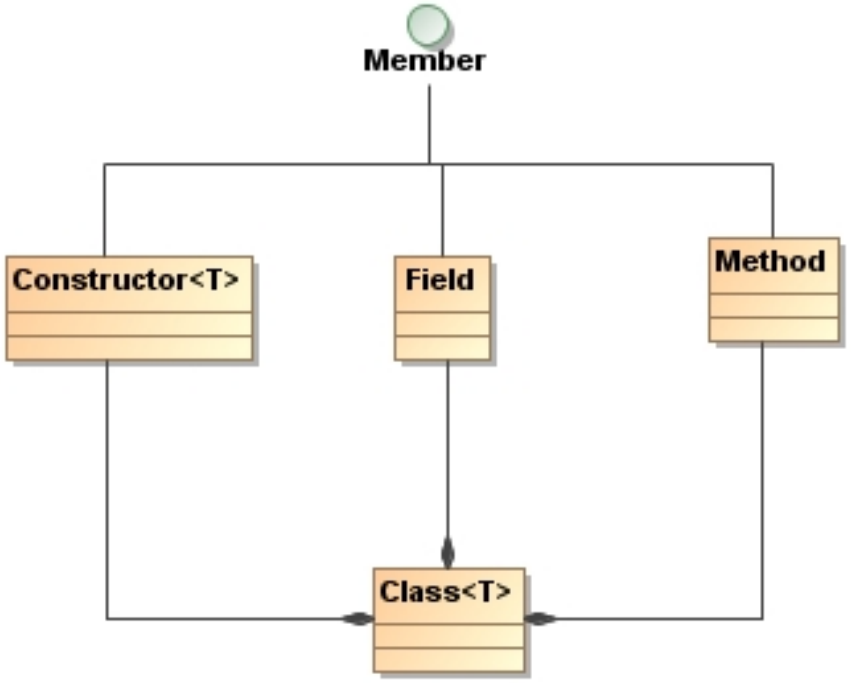
```
java.lang.Class<>
java.lang.reflect.Member
java.lang.reflect.Field
java.lang.reflect.Method
java.lang.reflect.Constructor
```



Introspección o Reflexión en Java

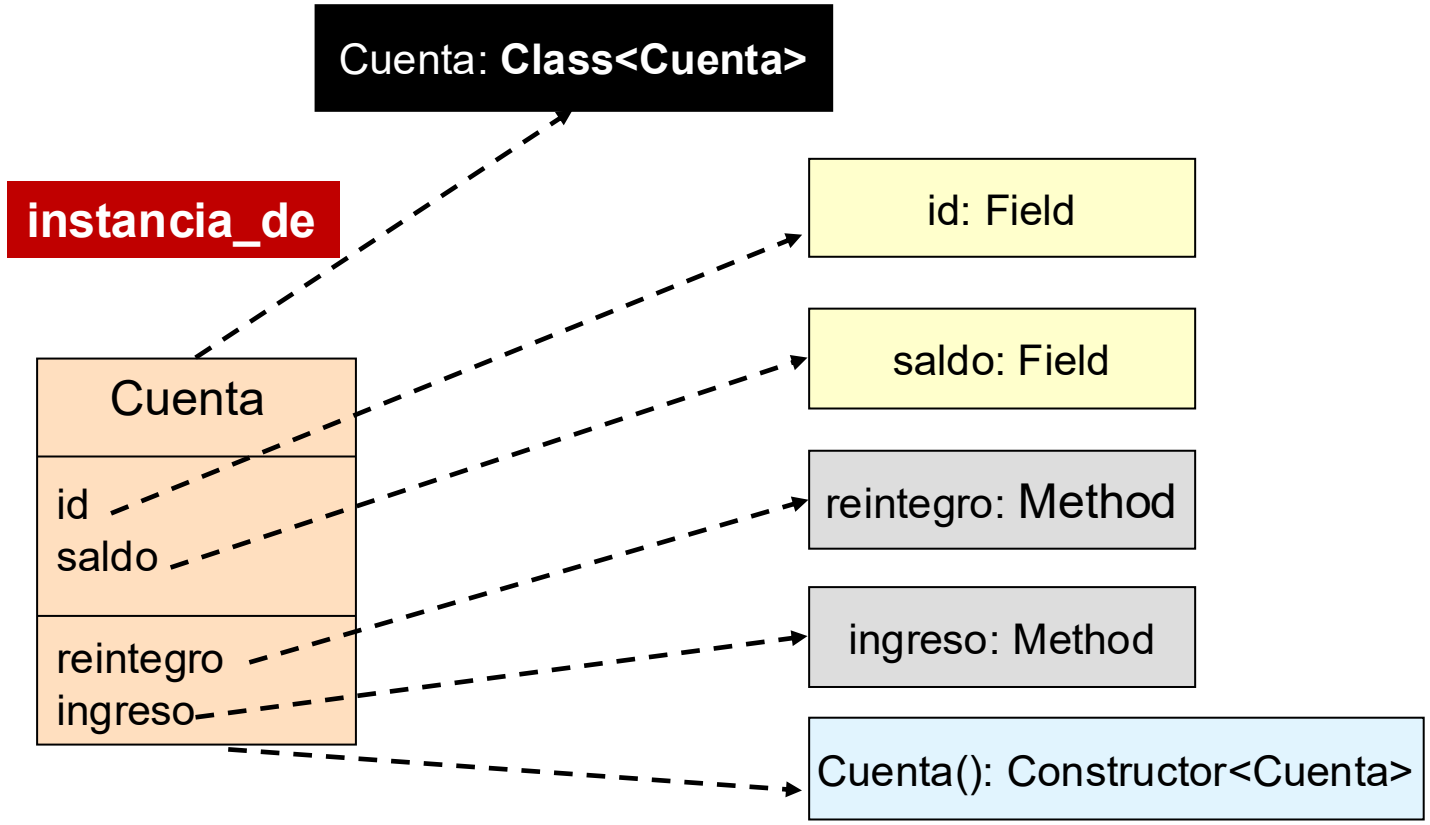
- Cada clase e interface es instancia de *Class<T>*.
  - Por ejemplo, la clase `Cuenta` es instancia de `Class<Cuenta>`
- Class<T> representa clases e incluye métodos que retornan información sobre las clases:
  - N*ombre (`getName`)*
  - *Campos (`getFields`, `getDeclaredFields`) y Metodos (`getMethods`, `getDeclaredMethods()`)*
  - *Superclase (`getSuperClass`)*
  - *Constructores (`getConstructors`, `getConstructor(Class<> .. parameters)`, `getDeclaredConstructors()`)*
  - *Interfaces que implementa (`getInterfaces`)*
  - *Paquete de la clase (`getPackage`)*
  - *Consultar tipo de clase (isPrimitive, isEnum, isInterface,..)*
  - *…*

Introspección o Reflexión en Java



- ***Member***
  - *getName, getDeclaringClass,..*
- ***Field*** *representa atributos (implementa Member):*
  - *getType, get(Object o), set(Object o),..*
- ***Method*** *representa métodos (implementa Member):*
  - *getName, getReturnType, getParameterTypes,..*

Introspección o Reflexión en Java



Introspección o Reflexión en Java

`Class<T>.forName(String className)`

Retorna la metaclase (instancia de `Class<T>`) de la clase cuyo nombre es dado como argumento.

`newInstance()` *(deprecated en Java 9)*

Método de instancia de Class<T> que retorna una instancia de la clase representada por el objeto receptor de tipo Class<T>.

```
String nombreClase; //path de la clase

Class<?>.forName(nomClase).newInstance()
```

Java <=8

```
Class<?>.forName(nomClase).
    getDeclaredConstructor().newInstance()
```

Java >=9

Introspección o Reflexión en Java

Formas de obtener una instancia de Class<T>, esto es una **metaclase.**

1. **getClass()** en clase Object

```
Cuenta oc;
..
Class<Cuenta> c = oc.getClass();
```

2. **forName()** en clase Class<T>

```
Class<Cuenta> c = Class<?>.forName ("tds.AppMusic");
```

3. Operador **class**

```
Class<Cuenta> c = Cuenta.class
Class<String> c = String.class
```

4. Métodos que retornan objetos de tipo Class<T>

```
Class<?> c = javax.swing.JButton.class.getSuperclass();
```

Ejemplo de Introspección. “Dada una clase, imprimir los nombres de sus campos y métodos.”
“El Lenguaje de Programación Java, K. Arnold et al., 3ª edición, Addison-Wesley, 2001.

```
package tds.reflexion.examples;
import java.lang.reflect.*;
import java.util.Arrays;

public class EjemploReflexion {
    public static void main (String[] args) {
        try {
            Class<?> c = Class.forName(args[0]);
            System.out.println(c);
            printMiembros (c.getFields());
            printMiembros (c.getConstructors());
            printMiembros (c.getMethods());
        }
        catch (ClassNotFoundException e)  {
            System.out.println("Clase desconocida: " + args[0]);
        }
    }
```

Ejemplo de Introspección. “Dada una clase, imprimir los nombres de sus campos y métodos.”
“El Lenguaje de Programación Java, K. Arnold et al., 3ª edición, Addison-Wesley, 2001.

```
private static void printMiembros(Member[] miembros) {
    // imprimir miembros excepto los miembros de Object
    Arrays.asList(miembros).stream()
          .filter(m -> m.getDeclaringClass() != Object.class)
          .map(m-> m.getName())
          .forEach(s -> System.out.println(s));
 }
}
```

Ejemplo: Invocar un método Method

- *getMethod()* retorna un método (instancia de *Method*) a partir del nombre del método y los tipos de sus parámetros.
- *invoke() invoca a un método (objeto receptor de la invocación) y tiene como parámetros el objeto receptor del método y los argumentos.*

```
public static void main(String[] args) {
   try {
     Class<Cuenta> classCuenta = Cuenta.class;
     Method m = classCuenta.getMethod("ingreso", new Class[] {int.class});
     Cuenta cu = new Cuenta("jesusGM");
     m.invoke(cu, new Object[]{new Integer(1000)});
     m.invoke(cu, new Object[]{new Integer(400)});
     System.out.println(cu.saldo);
   } catch (NoSuchMethodException e) {}
     catch (InvocationTargetException e) {}
     catch (SecurityException e) {}
}
```

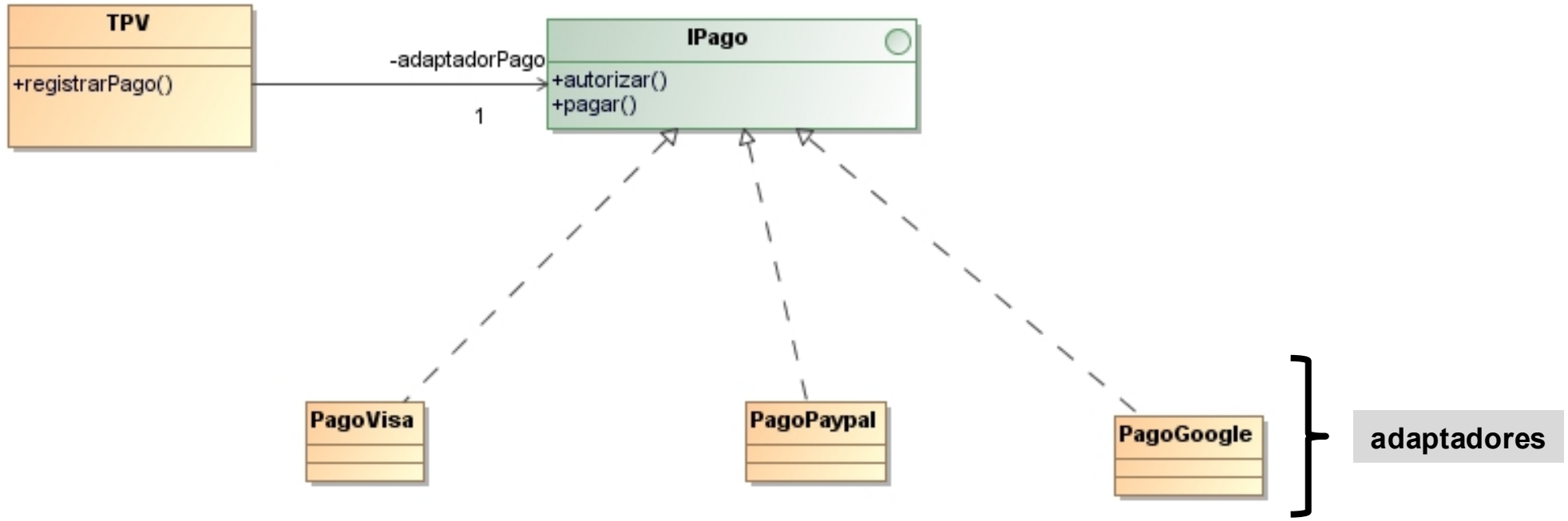
Item 53 en Effective Java, Joshua Bloch, 2ª ed., Addison-Wesley, 2008.

"En resumen, la reflexión es una utilidad muy potente que es requerida para algunas tareas de programación sofisticadas, pero que tiene muchas desventajas. Si estás escribiendo un programa que debe manipular clases desconocidas en tiempo de compilación, deberías, en todo lo posible, usar reflexión sólo para instanciar objetos y acceder a esos objetos utilizando alguna interfaz o superclase conocida en tiempo de compilación"

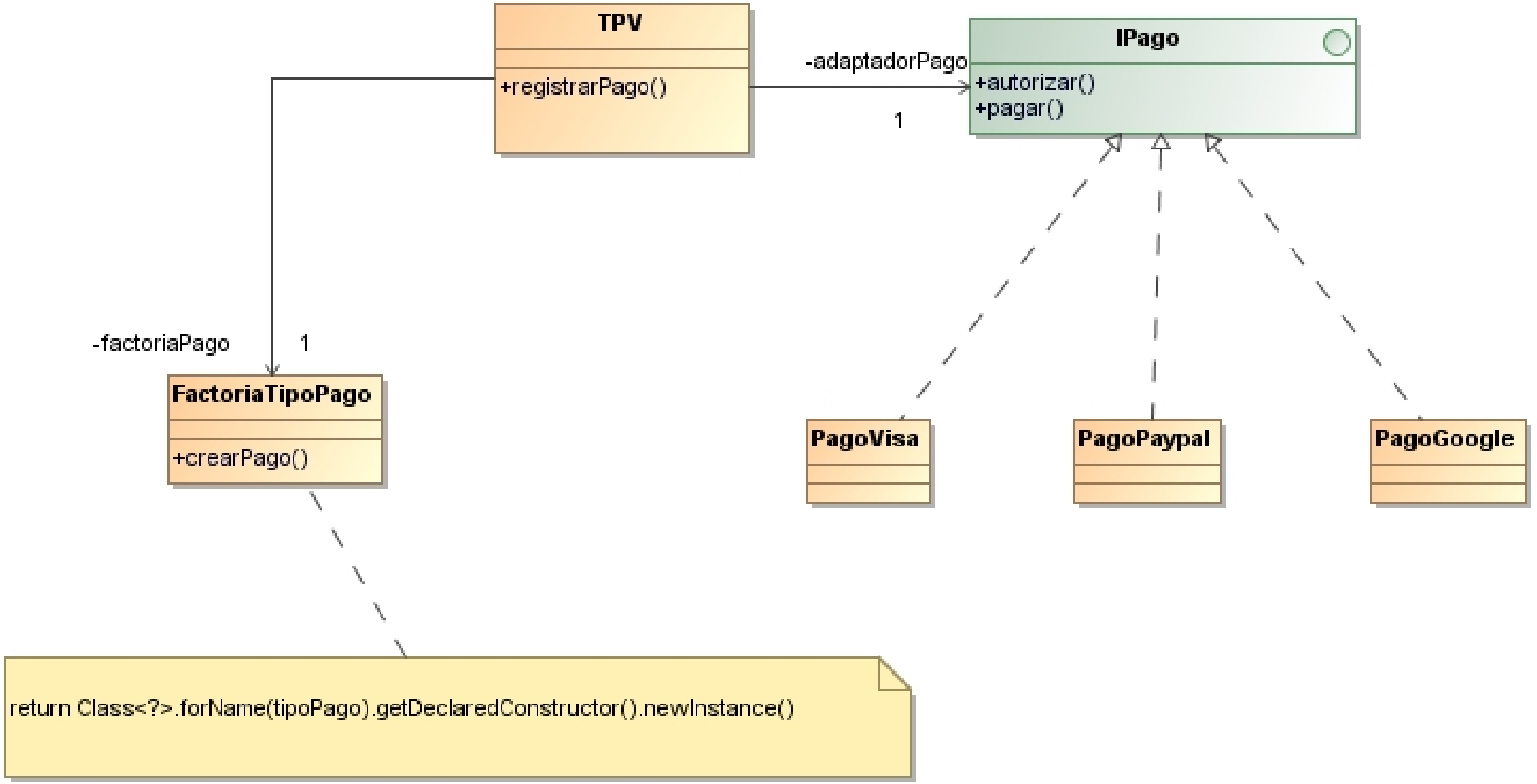
Desventajas:

- Se pierden beneficios del control de tipos en compilación, errores en tiempo de ejecución.
- Código verboso y tedioso de escribir.

¿Cómo crear un adaptador de tipo de pago sin especificar la clase en TPV?

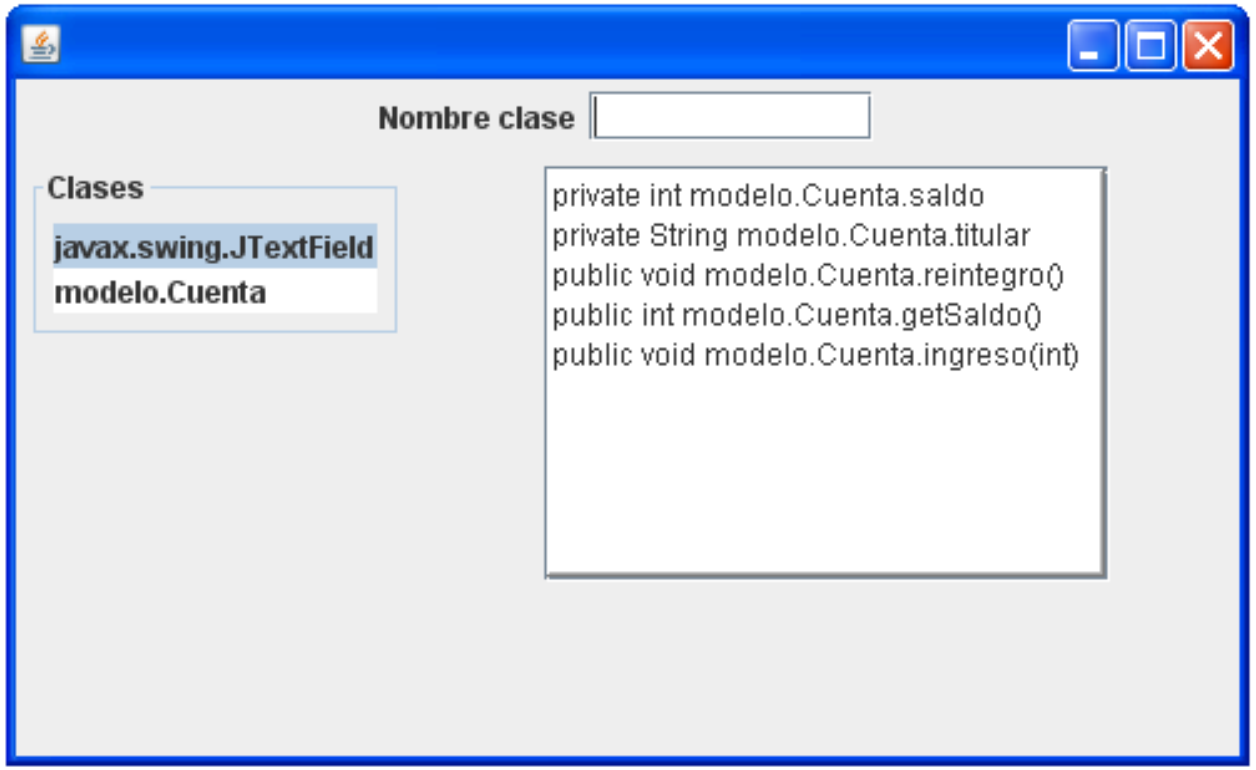


Ejemplo de introspección (*Método factoría*, revisado en patrones): **Clase Factoría**



Ejercicio 1. Introspección Java

Crea una ventana como la que se muestra abajo para mostrar los métodos y campos declarados en una clase dada. En el campo de texto de arriba se introduce un nombre de clase que se añade a la lista de la izquierda y se muestran sus campos y métodos (a excepción de los que hereda de `Object`) en el área de texto de la derecha, al añadir una nueva clase o al seleccionar un elemento de la lista.



Ejercicio 2. Introspección Java

La clase `Collections` proporciona el método `sort(lista, comparador)` que ordena la lista que se pasa como primer argumento (`List<T>`) utilizando como criterio de ordenación el comparador que se pasa como segundo argumento (instancia de una clase que implementa la interfaz `Comparator<T>`).

Dada una clase C, se desea ordenar listas de objetos de C de acuerdo al valor de uno de sus campos que debe ser de tipo dato primitivo, por ejemplo una lista de objetos `Usuario` por su campo `dni` o `nombre` de tipo String. Utiliza la introspección de Java para idear una forma que evite tener que crear una clase que implemente `Comparator<T>` por cada par <tipo de objeto, campo>, sino una única implementación que sea independiente de un par concreto <clase y campo>.

Este ejercicio estaba planteado para Java 7 y puede reconsiderarse también para Java 8.

java.util
Class Collections

```
sort(List<T> list, Comparator<? super T> c)
```

Ordena la lista especificada de acuerdo al orden establecido por el comparator establecido.

java.util
Interface Comparator<T>

```
compare(T o1, T o2)
```

Compara argumentos para ordenar

Ejercicio

```
class Cliente {
   private String dni;
   private String nombre;
   private int sueldo;
   // getters and setters
}
```

Ordenar **`ArrayList<Cliente>`** por `dni`, o `nombre`, o sueldo, ..

```
class Cuenta {
   private String ccc;
   private String titular;
   private int saldo;
   // getters and setters
}
```

Ordenar **`ArrayList<Cuenta>`** por `ccc`, o `titular`,..

Crear una clase que implemente `Comparator<T>` que evite crear un comparador concreto por cada par clase/campo, por ejemplo, `Cliente/dni`.
Esto es, que esté parametrizada por esa información.

```
// Ordenar por dni
usuarios.sort(new ReflectiveComparator<> ("dni"));
// Ordenar por nombre
usuarios.sort(new ReflectiveComparator<> ("nombre"));
```

```
import java.lang.reflect.Method;
import java.util.Comparator;

public class ReflectiveComparator<T> implements Comparator<T> {
     private String propertyName;

     public ReflectiveComparator(String propertyName) {
            this.propertyName = propertyName;
     }

     private Object getPropertyValue(T obj, String propertyName) {
            try {
                  String getterName = "get" + propertyName.substring(0, 1)
                        .toUpperCase() + propertyName.substring(1);
                  Method getterMethod = obj.getClass().getMethod(getterName);
                  return getterMethod.invoke(obj);
             }
             catch (Exception e) {
                     throw new RuntimeException("Error al obtener el valor de la
                                                propiedad usando reflexión", e);
            }
      }

      @Override
      public int compare(T o1, T o2) {
           Comparable value1 = (Comparable) getPropertyValue(o1, propertyName);
           Comparable value2 = (Comparable) getPropertyValue(o2, propertyName);
           return value1.compareTo(value2); }
       }
}
```